



Order Book Analyzer – Status Report 4

FINTECH 512 - Software Engineering for FinTech

Full team of five members for the group project (section 1)



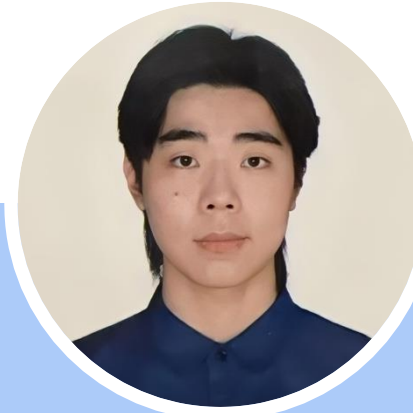
Rebeca Hechtman
NetId: rh398



Mike Lin
NetId: hl561



Mory Shi
NetId: cs836



Loki Luo
NetId: lz294



Alan Song
NetId: ys501

Agenda

- 01.** GitLab Repository and Issues
- 02.** Project Ideation
- 03.** User and Admin Interface Prototype
- 04.** Updated Design
- 05.** Status, Review and Planning

GitLab Repository and Issues

Team Project GitLab Repository

https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#

The screenshot shows a web browser displaying the GitLab interface for a repository named 'pred_market_data_platform' under the 'team-hotel' namespace. The browser's address bar shows the URL 'gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#'. The GitLab page has a sidebar on the left with a 'Project' section containing 'pred_market_data_platform' and a list of items: 'Pinned', 'Issues', and 'Merge requests'. Below this is a 'Manage' section with links for 'Plan', 'Code', 'Build', 'Secure', 'Deploy', 'Operate', 'Monitor', 'Analyze', and 'Settings'. At the bottom of the sidebar are 'What's new' and 'Help' links. The main content area shows the repository name 'pred_market_data_platform' with a 'Code' button. Below this is a message stating 'The repository for this project is empty' and instructions to clone or upload files. There are sections for 'Command line instructions', 'Configure your Git identity' (with 'Local' and 'Global' tabs), 'Git local setup' (with instructions to configure identity locally), 'Add files' (with instructions to push files using SSH or HTTPS), and 'Create a new repository' (with a 'git clone' command). On the right side, there is a 'Project information' section with an 'Invite your team' button and a list of actions: 'Upload File', 'New file', 'Add README', 'Add LICENSE', 'Add CHANGELOG', 'Add CONTRIBUTING', 'Set up CI/CD', 'Add Wiki', and 'Configure Integrations'. At the bottom right, it says 'Created on March 06, 2026'.

team-hotel / pred_market_data_platform

Search or go to...

Project

- pred_market_data_platform
- Pinned
- Issues
- Merge requests

Manage

- Plan
- Code
- Build
- Secure
- Deploy
- Operate
- Monitor
- Analyze
- Settings

What's new

Help

Collapse sidebar

team-hotel / pred_market_data_platform

pred_market_data_platform

Code

The repository for this project is empty

To get started, clone the repository or upload some files.

Command line instructions

You can also upload existing files from your computer using the instructions below.

Configure your Git identity

Get started with Git and learn how to configure it.

Local Global

Git local setup

Configure your Git identity locally to use it only for this project:

```
git config --local user.name "rebecca.hechtman@duke.edu"
git config --local user.email "rh398@duke.edu"
```

Add files

Push files to this repository using SSH or HTTPS. If you're unsure, we recommend SSH.

SSH HTTPS

How to use SSH keys?

Create a new repository

```
git clone git@gitlab.oit.duke.edu:team-hotel/pred_market_data_platform.git
```

Project information

Invite your team

Add members to this project and start collaborating with your team.

Invite members

Upload File

- New file
- Add README
- Add LICENSE
- Add CHANGELOG
- Add CONTRIBUTING
- Set up CI/CD
- Add Wiki
- Configure Integrations

Created on

March 06, 2026

Legend:

- Done
- In progress
- To be done

Project Requirements – Organization and Ideation

1. Organization

1. W1: List the requirements
2. W2: Refine the requirements

2. Gitlab

1. W1: Create Gitlab
2. W2: Update the issues (+ improve wording)
3. W2: Understand Gitlab SAST Tooling

3. System Architecture diagrams (C4)

1. W1: Create System Architecture
2. W2: Create using C4 model
3. W3 - Refine diagrams

4. Virtual Machines (current version of site running on a Duke VCM)

1. W1: Apply VCM
2. Check how to share the VCM with the group
3. Share the VCM port with the group
4. Deploy the app (Gitlab -> virtual machine)
5. Automatically starting

5. Security Analysis - DFD threat model

Create a Data Flow Diagram (DFD) for your BigBucks system, identify assets, actors, and threats, and perform a STRIDE analysis of DFD

1. W2: 1st overview draft
2. W3: Refinement and detailing

6. Users and Administrative Functionality:

1. W1: initial user's type ideation
2. W2: Admin's ideation
3. Detailing of user
4. Detailing of admin

7. Data service

1. W1: data design ideation
2. W1: data flow ideation (Polymarket API)
3. W2: testing ideation
4. W4: API to connect full stack with data service – Mory

8. Full stack

1. W2-W3: Data design of user ideation
2. W4: Data design of admin ideation - Alan
3. W2: testing ideation

9. Charting (3 or more + custom interaction)

1. W3: Ideation + paper prototype
2. Refine + Include custom interaction
3. Document with definitions

10. Paper prototypes

1. W1 – W3: User Interface – Rebeca
1- Home page for unauthenticated users + User authentication page; 2- New user registration page, 3- "Authenticated" home page for regular users, 4- Dashboard of relevant information, 5- User Information, 6- "Analysis" page that allows the user to make an informed decision to buy/sell a particular product, item, and or service + Allow the user to buy/sell a product, item, or service, 7- Analysis page of the user's current data, 8- History screen to show past transactions, 9- Reporting functionality
2. W4-W5: Tabs for admin functionality: View all users, Disable user accounts, Reset user passwords, View 'system' transactions, Perform some sort of analysis across all accounts - Rebeca

Project Requirements – GitLab and Coding

Legend:

- Done
- In progress
- To be done

11. Organization

1. W2: Setup the overall project repo structure – Rebeca
2. W1-W6: Keep GitLab issues up-to-date - Alan
3. W3: Talk about merge requests - All

12. Full Stack

1. Front-end design
 1. W3-W4: CSS for styling – Loki
 2. W3-W4: HTML (display) + JavaScript (interaction) for all the page prototypes of user interface (for the pages that need data, use dummy data) – Loki
 3. Connect with back-end API
 4. Update with real data
 5. HTML tabs for admin functionality
2. Back-end design:
 1. W4: JavaScript to connect front end with the API - Alan
 2. Data base with user roles and permissions
 3. W4: OAuth2 Authentication Integration - Alan
3. Database design:
 1. W4: for user (PostgreSQL) - Alan
4. Tests suite (including DFD created)
 1. Unit tests
5. Logging and Monitoring
 1. Understand
 2. Design
 3. Implement

13. CI/CD – YAML file

1. Build release software on every commit
2. Runs test suite on every commit
3. Runs Gitlab SAST Tooling
4. Automatically deploy your software to your VCM

14. Data Service Design

1. W2: API to connect data service with Polymarket (Data Integration using a REST/JSON web API)
2. W4: API to connect full stack with data service – Mory
3. W2: Message queue
4. W2: Database (PostgreSQL) implementation
5. W4: Update transformation – Mike
6. Resampling of transformation
7. W3-W5: Tests suite; also use DFD created – Mike

15. Containerize Application

1. Create docker files to create an image of the application
2. Create a docker compose file to deploy and execute image

16. Security Analysis – run tests against the software

1. Run Semgrep analyzer against your code base
2. Runs OWASP Zed Attack Proxy

Project Requirements – Detail Front End

Legend:

- Done
- In progress
- To be done

1. CSS for styling
2. HTML for all the page prototypes (for the pages that need data, using dummy data for now)
 1. Home page for unauthenticated users
 2. User authentication page
 3. New user registration page
 4. "Authenticated" home page for regular users. Dashboard of relevant information.
 5. "Analysis" page that allows the user to make an informed decision to buy/sell a particular product, item, and or service + Allow the user to buy/sell a product, item, or service
 6. Analysis page of the user's current data
 7. History screen to show past transactions
 8. Reporting functionality
3. HTML tabs for admin functionality:
 1. View all users, Disable user accounts
 2. Reset user passwords
 3. View 'system' transactions
 4. Perform some sort of analysis across all accounts

Project Issues

(updated on April 3)

Link to the issue page: https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform/-/issues?sort=created_asc&state=opened&last_page_size=20&page_before=eyJjcmVhdGVkX2F0IjoiMjAyNi0wMy0xOSAxMDozNTozMS41MDU4MzcxMDAgLTA0MDAiLCJpZCI6IjQ1NTQyIn0		
✔ initial user's type ideation	team-hotel/pred_market_data_platform#41 · created 2 weeks ago by yinglun.song@duke.edu	Done updated 1 week ago
✔ Database design for users	team-hotel/pred_market_data_platform#42 · created 2 weeks ago by yinglun.song@duke.edu	Done updated 1 week ago
📁 Data service	team-hotel/pred_market_data_platform#43 · created 2 weeks ago by yinglun.song@duke.edu	Current in Progress updated 1 week ago
✔ data design ideation	team-hotel/pred_market_data_platform#44 · created 2 weeks ago by yinglun.song@duke.edu	Done updated 1 week ago
✔ data flow ideation (APIs)	team-hotel/pred_market_data_platform#45 · created 2 weeks ago by yinglun.song@duke.edu	Done updated 1 week ago
✔ testing ideation	team-hotel/pred_market_data_platform#46 · created 2 weeks ago by yinglun.song@duke.edu	Done updated 13 minutes ago
📁 Full stack	team-hotel/pred_market_data_platform#47 · created 2 weeks ago by yinglun.song@duke.edu	Current in Progress updated 1 week ago
✔ data design ideation	team-hotel/pred_market_data_platform#48 · created 2 weeks ago by yinglun.song@duke.edu	Done updated 1 week ago
✔ testing ideation	team-hotel/pred_market_data_platform#49 · created 2 weeks ago by yinglun.song@duke.edu	Done updated 1 week ago
📁 Charting (3 or more + custom interaction)	team-hotel/pred_market_data_platform#50 · created 2 weeks ago by yinglun.song@duke.edu	To Do updated 10 minutes ago

*Some of the issues that are created

Project Ideation

Project's 'elevator pitch'



Bloomberg of Prediction Markets



Prediction markets, where people bet real money on the outcomes of future events (like whether Bitcoin will hit \$110,000 this year), are exploding in popularity. But making sense of all that betting activity can be frustrating. The raw data behind these markets is messy, disorganized, and hard to easily analyze.

Order Book Analyzer cuts through that noise. Think of us as the Bloomberg for prediction markets. We take massive amounts of complicated, second-by-second betting data and translate it into clear, powerful dashboards and insights. Instead of wrestling with messy data, our platform gives you the exact trends and historical information you need to see exactly where the smart money is going.

Project Scope Disclaimer: This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1). Future market expansions remain under evaluation based on technical complexity and timeline constraints.

The Lean Canvas

Designed for:

Order Book Analyzer

Designed by:

Team Hotel

Date:

03/16/2026

Version:

2.0

Problem  1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps. 2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort. 3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction.	Solution  A website provides Insights & Dashboards: Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)	Unique Value Prop.  High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)	Unfair Advantage  1. Curated Historical Data Archive: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants. 2. Domain & Technical Expertise	Customer Segments  - Mid and small size Hedge Funds - Sophisticated algo traders who wants to trade in prediction market
Existing Alternatives  - Eagle Alpha - Dune Analytics - DIY engineering - Basic Polymarket Dashboards	Key Metrics  Growth: # of users - Retention - API call volumes - Daily Active User System Health: - Data Quality Score	High-Level Concept  - Yahoo Finance for Prediction Market - Bloomberg for Prediction Market	Channels  - Social Media (LinkedIn, X, Reddit, Red Note) - Organic Growth - Search Engine - Ads	Early Adopters  Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.
Cost Structure  Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage. Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets. Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation. Compliance: Legal frameworks and terms of service management.			Revenue Streams  Revenue Model: Freemium <u>Freemium Model:</u> Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth. <u>Key Account (KA) Annual Licenses:</u> High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds. <u>Value-Added Services (Overage):</u> Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.	

Changes we've made since the project started

The Lean Canvas		Designed for: Order Book Analyzer	Designed by: Team Hotel	Date: 03/16/2026	Version: 2.0
Problem  1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps. 2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort. 3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction.	Solution  A website provides Insights & Dashboards: Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)	Unique Value Prop.  High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)	Unfair Advantage  1. Curated Historical Data Archive: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants. 2. Domain & Technical Expertise	Customer Segments  1. Mid and small size Hedge Funds 2. Sophisticated algo traders who wants to trade in prediction market	
Existing Alternatives  1. Eagle Alpha 2. Dune Analytics 3. DIY engineering 4. Basic Polymarket Dashboards	Key Metrics  Growth: 1. # of users 2. Retention 3. API call volumes 4. Daily Active User System Health: 1. Data Quality Score	High-Level Concept  1. Yahoo Finance for Prediction Market 2. Bloomberg for Prediction Market	Channels  1. Social Media (LinkedIn, X, Reddit, Red Note) 2. Organic Growth 3. Search Engine 4. Ads	Early Adopters  Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.	
Cost Structure  Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage. Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets. Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation. Compliance: Legal frameworks and terms of service management.		Revenue Streams  Revenue Model: Freemium Freemium Model: Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth. Key Account (KA) Annual Licenses: High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds. Value-Added Services (Overage): Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.			

Lean Canvas is adapted from The Business Model Canvas (www.businessmodelgeneration.com/canvas). PowerPoint Implementation by: Neos Chronos Limited (<https://neoschronos.com>). License: CC BY-SA 3.0

Scope Restriction:

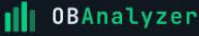
- This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1).
- Future market expansions remain under evaluation based on technical complexity and timeline constraints.

User and Admin Interface Prototype

Prototype Screens

1- Home Page

for unauthenticated users + User authentication page



[Create Account](#) [Sign In](#)

Order Book Analyzer

for Prediction Markets

Capture, store and replay real-time market data from Polymarket and BTC/USD. Built for backtesting and algorithmic strategy development.

[Create Free Account](#)

Real-time
WebSocket ingestion

Replay
Tick-level playback

Depth
1s & 5s intervals

Welcome back

Sign in to access your analytics.

 Continue with Google

 Continue with GitHub

OR

EMAIL

 trader@example.com

PASSWORD



☒ Remember me [Forgot password?](#)

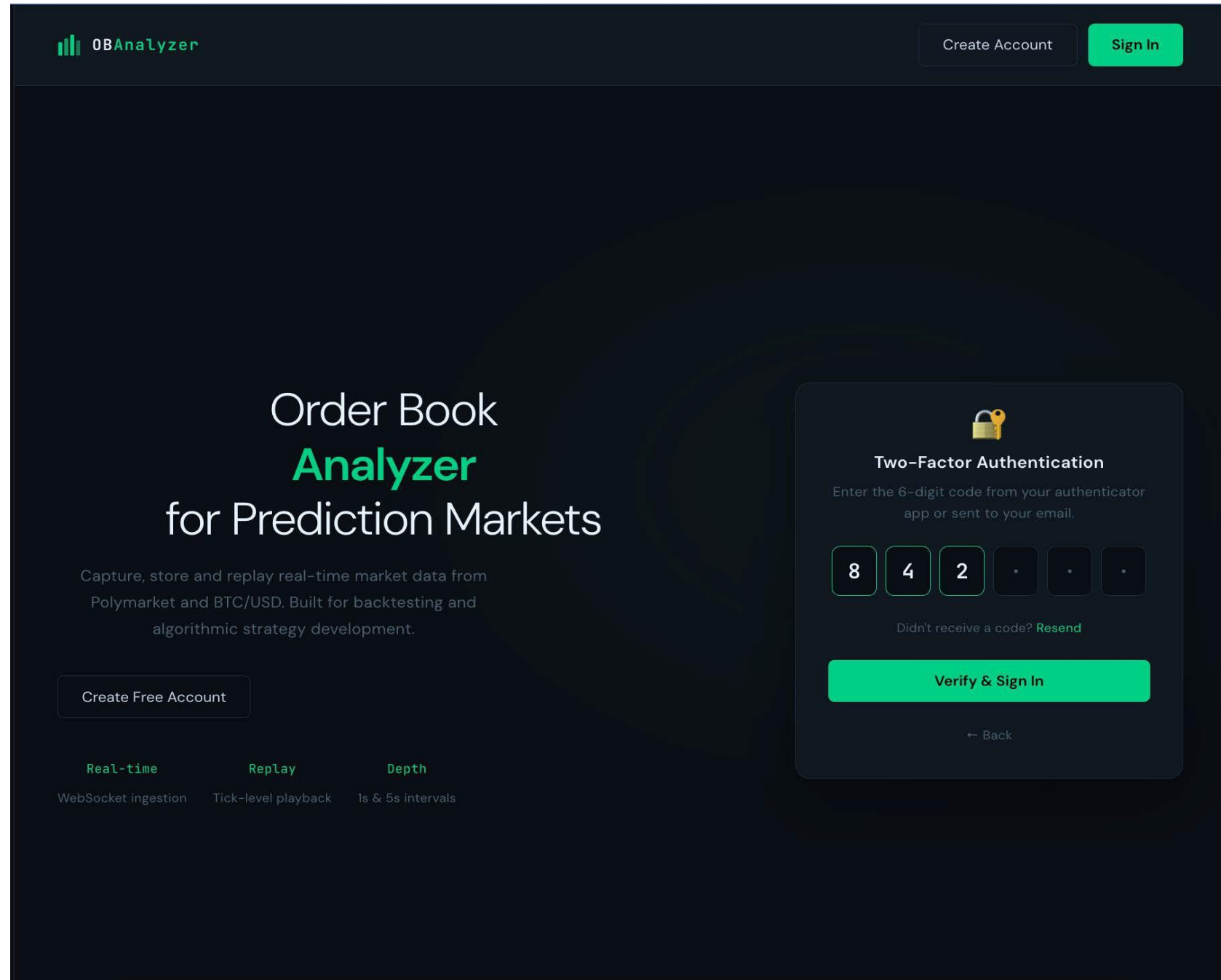
[Sign In](#)

No account? [Create one free](#)

 256-bit SSL · SOC 2 Type II

Prototype Screens

1b- User Authentication



Prototype Screens

2- New user registration page

OBAnalyzer

Already have an account? [Sign In](#)

FREE ACCESS

Start analyzing
prediction markets

Get instant access to BTC order book replays, binary option depth charts, and historical snapshot data.

 **Dashboard**
Visualize BTC prices & 15-min binary options

 **Replay Engine**
Full order book playback at any timestamp

 **Depth Charts**
Cumulative depth at tick, 1s and 5s

Create Account

Set up your profile in seconds

FIRST NAME

LAST NAME

Jane

Doe

EMAIL

trader@example.com

USERNAME

@handle

PASSWORD

Min. 8 characters

CONFIRM PASSWORD

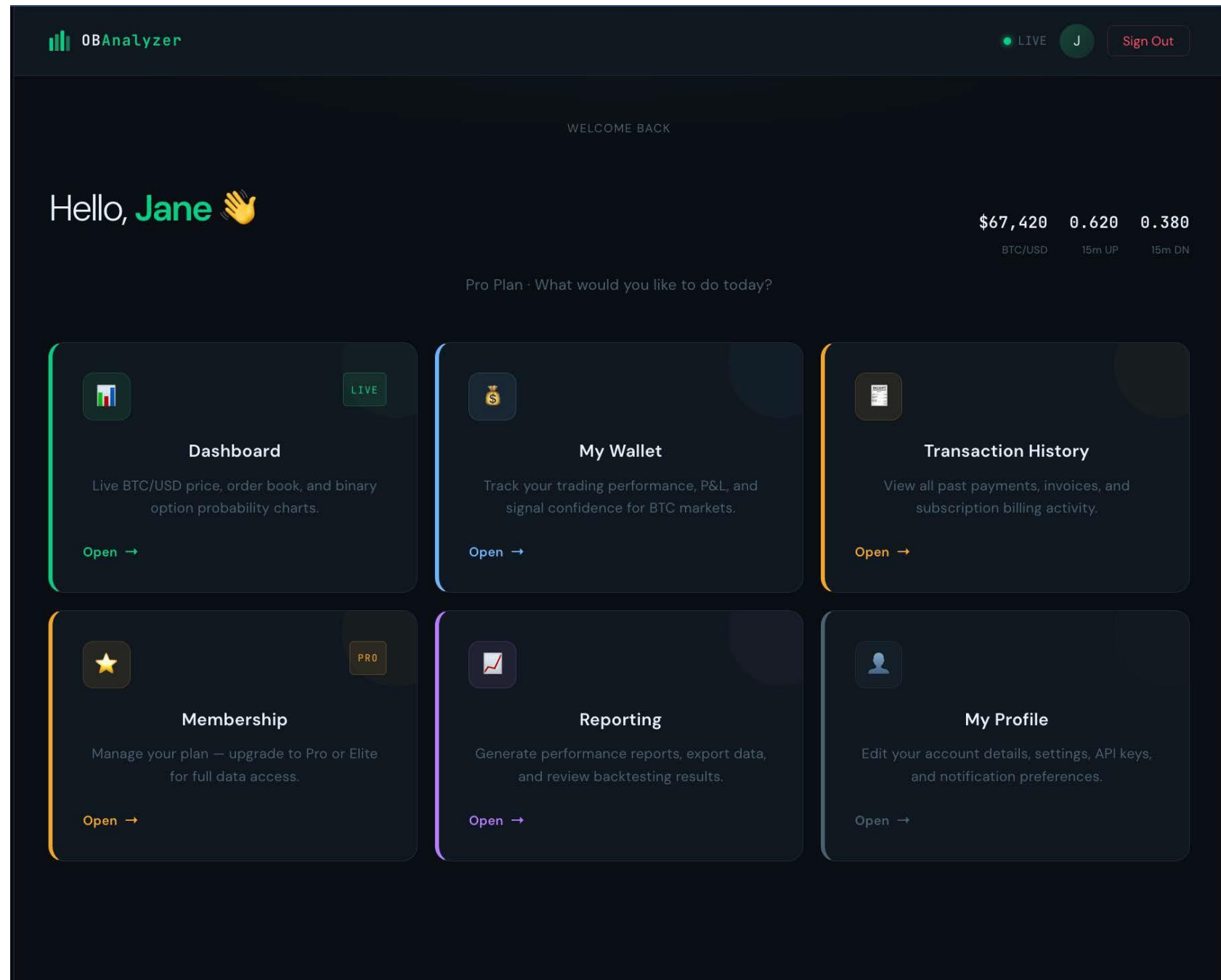
Repeat password

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#)

Create Account →

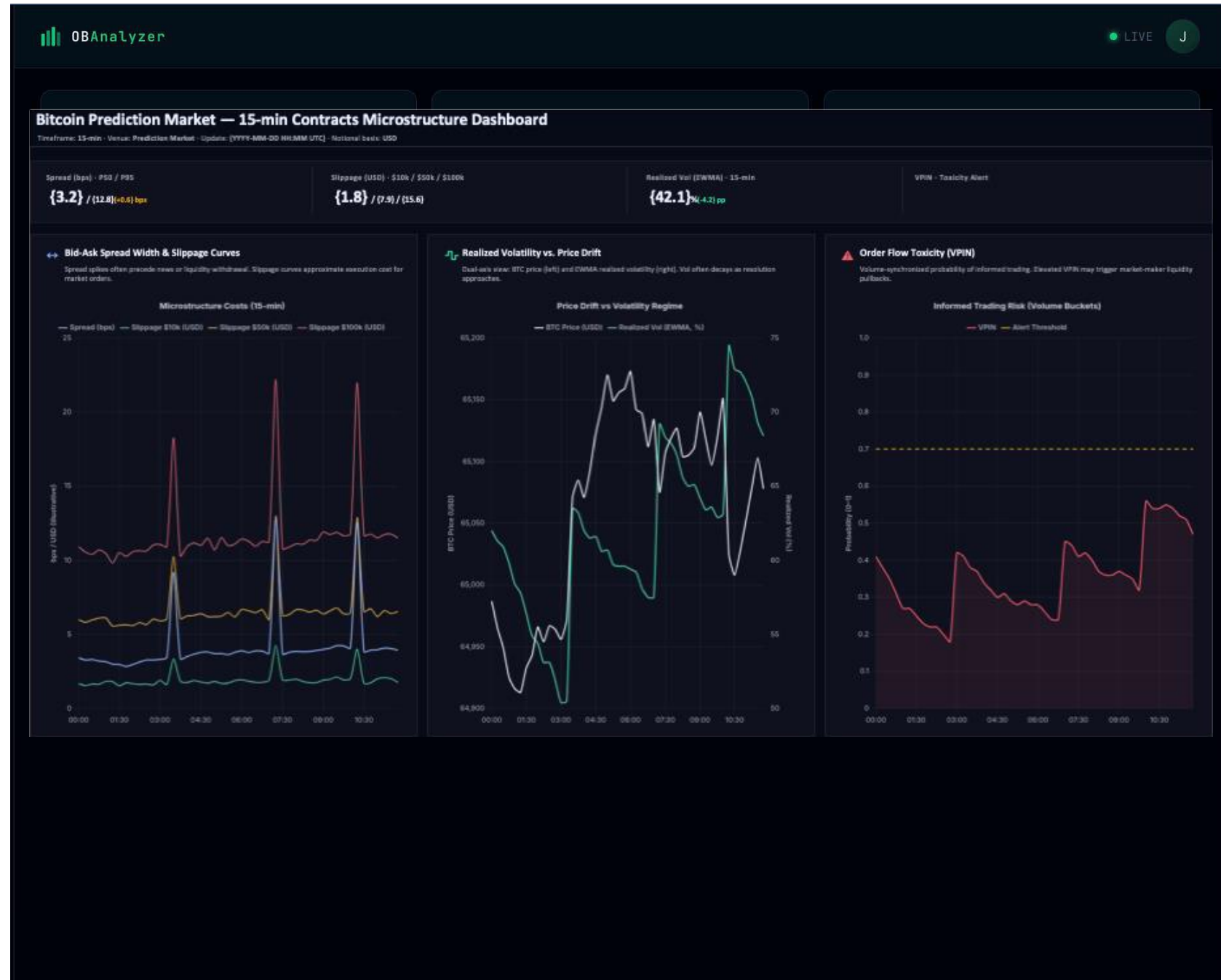
Prototype Screens

3- Authenticated home page for regular users



Prototype Screens

4- Dashboard of relevant information



Charting – v1 Ideation

Bitcoin Prediction Market — 15-min Contracts Microstructure Dashboard

Timeframe: 15-min · Venue: Prediction Market · Update: {YYYY-MM-DD HH:MM UTC} · Notional basis: USD

Spread (bps) · P50 / P95

{3.2} / {12.8}{+0.6} bps

Slippage (USD) · \$10k / \$50k / \$100k

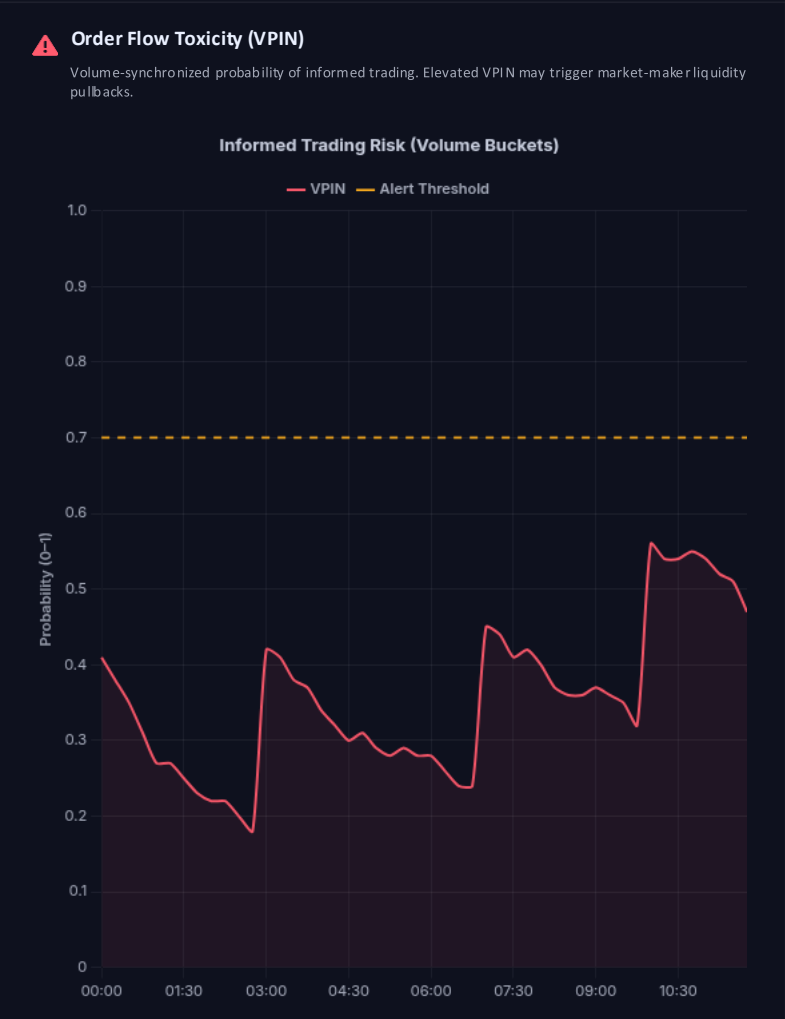
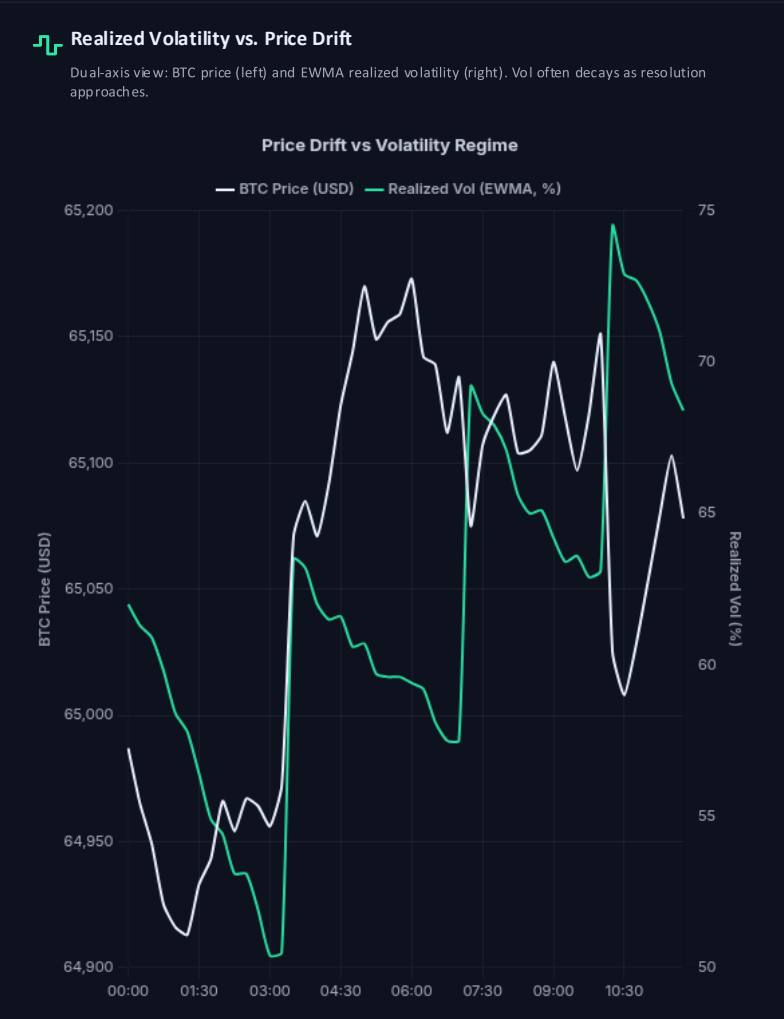
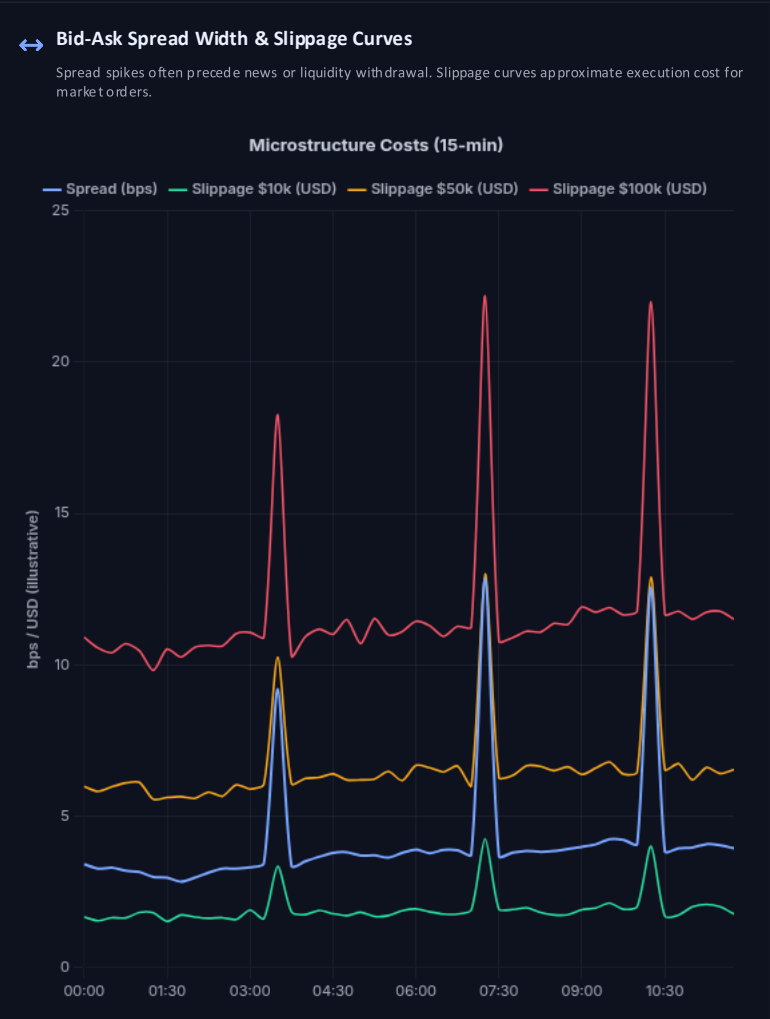
{1.8} / {7.9} / {15.6}

Realized Vol (EWMA) · 15-min

{42.1}%{-4.2} pp

VPIN · Toxicity Alert

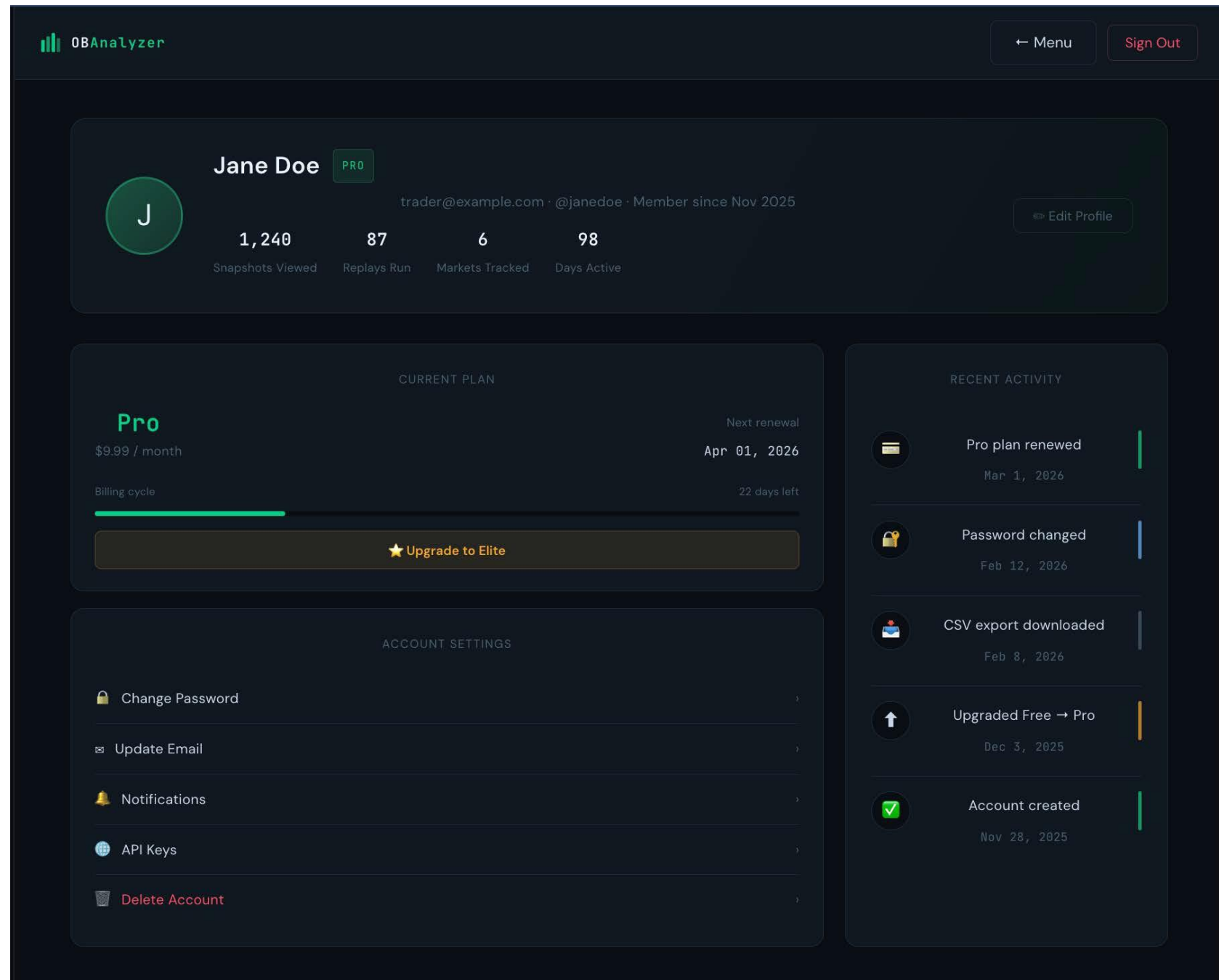
{0.62} HIGH(threshold: 0.70)



Method notes: Spread = (Ask - Bid) / Mid (bps). Slippage curves are theoretical cost estimates (illustrative) calibrated to instantaneous depth. Realized vol shown as EWMA of 1-min returns annualized (placeholder). VPIN computed on volume buckets (placeholder). Replace placeholders with your venue's tick/orderbook feed.

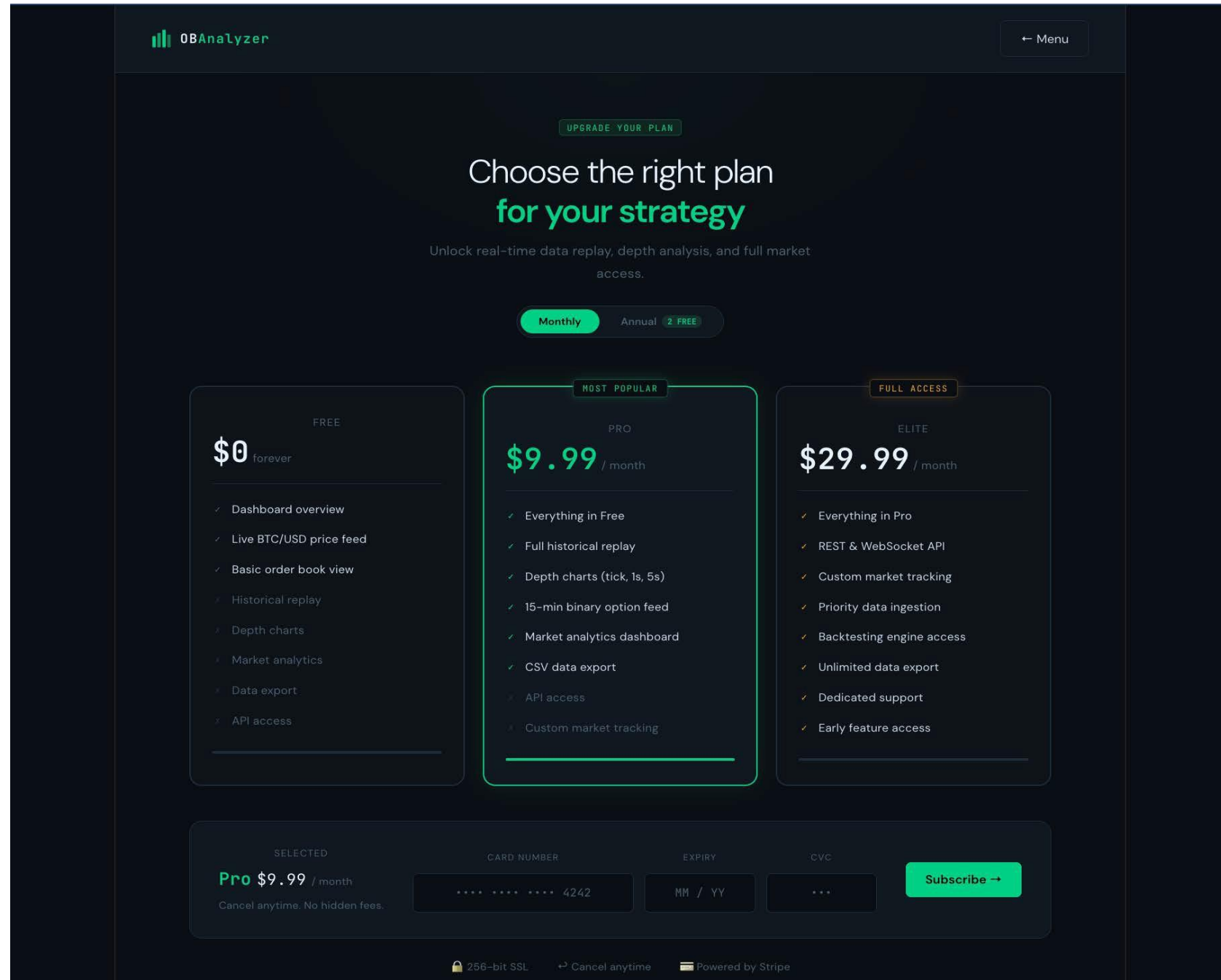
Prototype Screens

5- User information



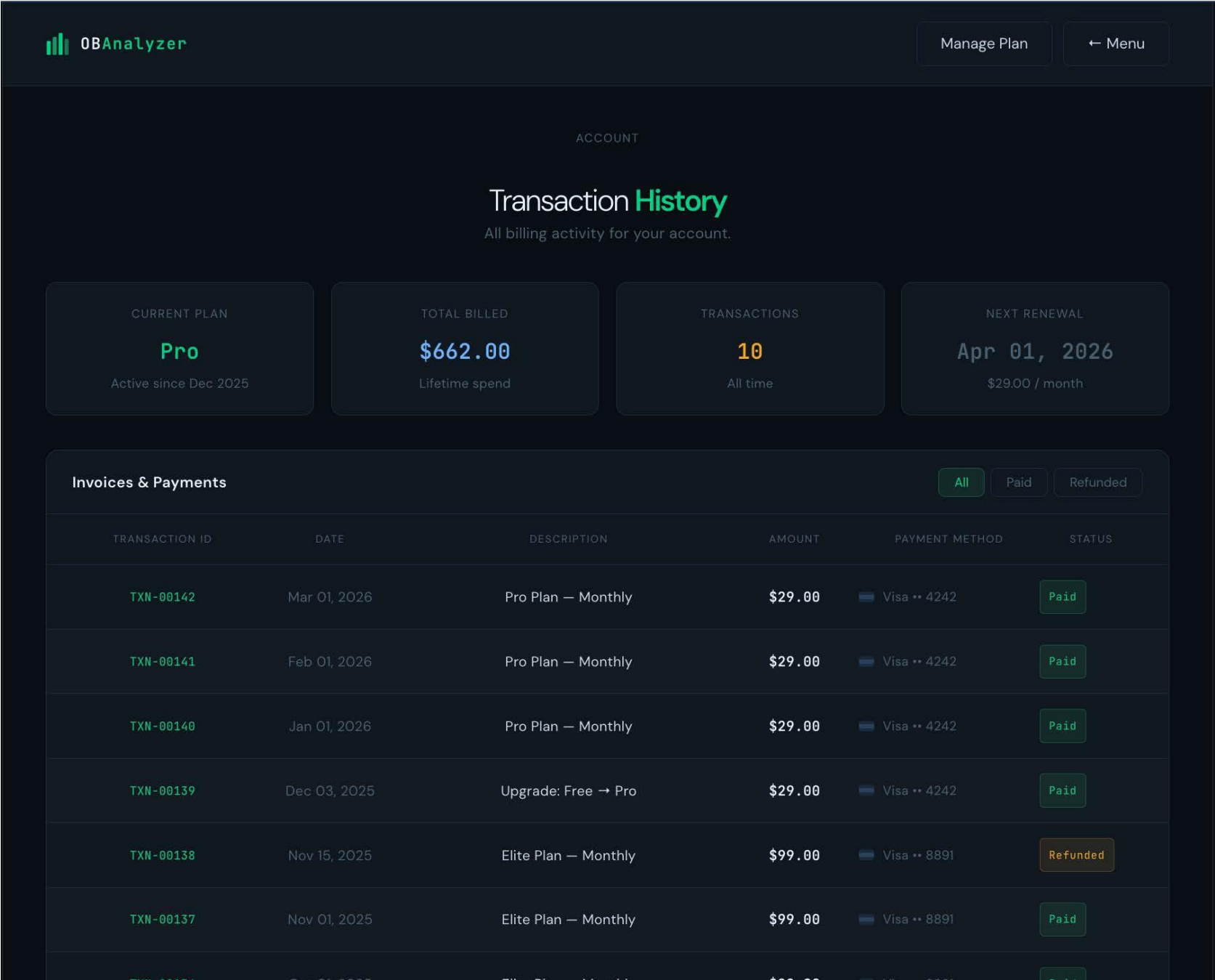
Prototype Screens

6- Buy product / service



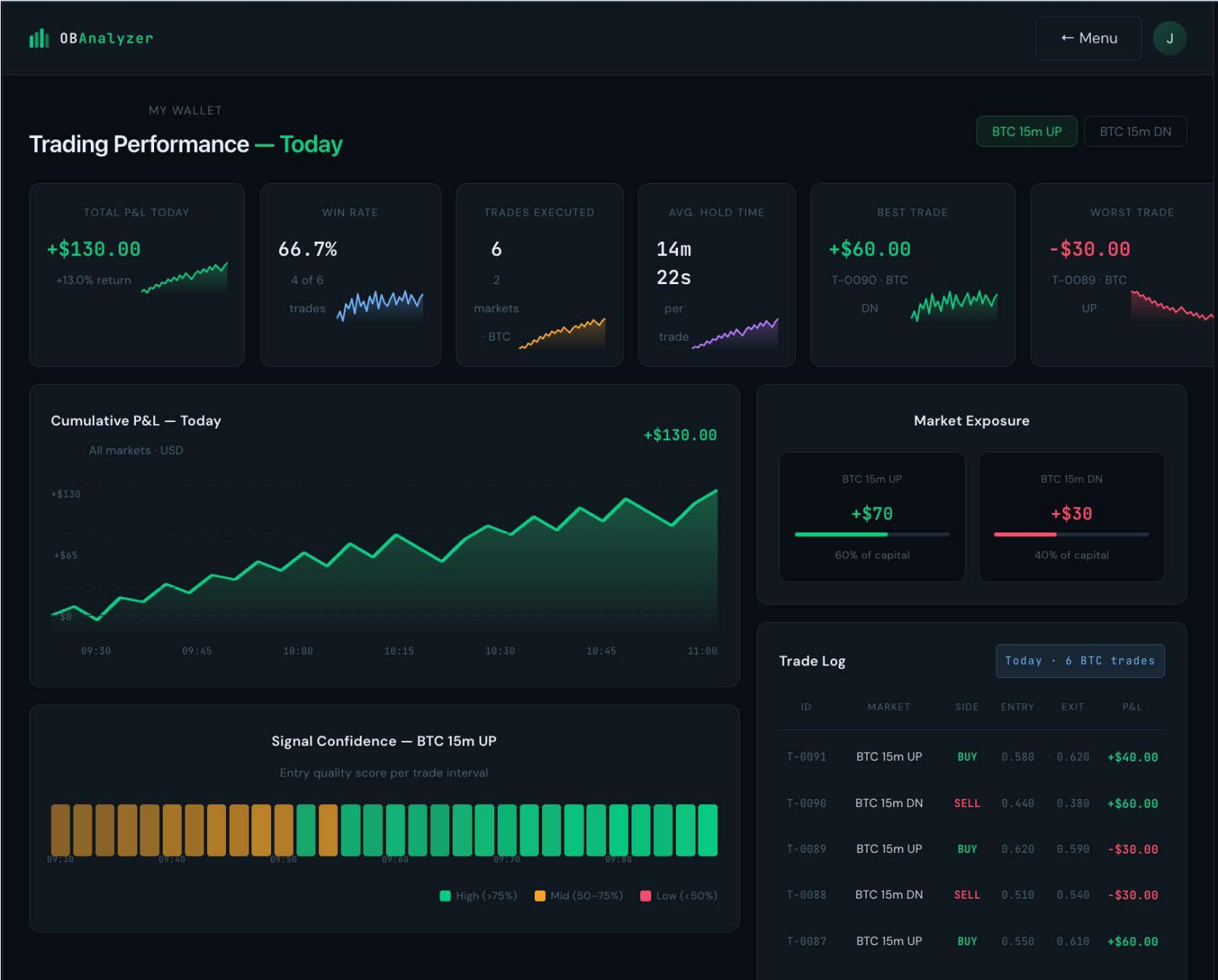
Prototype Screens

7- History screen to show past transactions



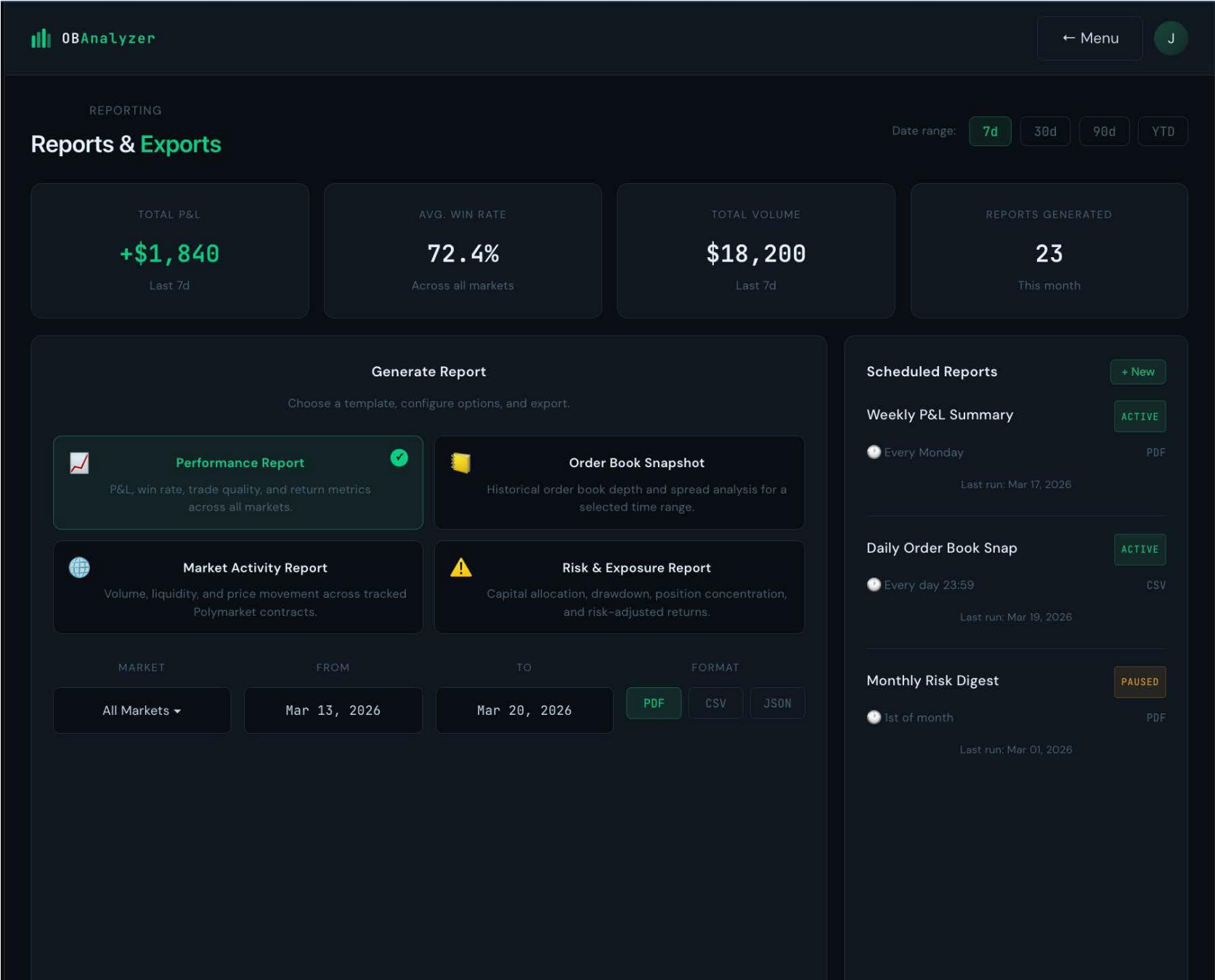
Prototype Screens

8- Analysis page of the user's current data



Prototype Screens

9 - Reporting functionality



Prototype Screens

10 – Admin

View All Users

OBAnalyzer

ADMIN

← Menu

A

All Users

Disable Accounts

Reset Passwords

System Transactions

Platform Analysis

TOTAL USERS

8

ACTIVE

6

DISABLED

2

ELITE MEMBERS

3

PRO MEMBERS

3

FREE USERS

2

User Directory

Search name or email...

ID	NAME	EMAIL	PLAN	JOINED	LAST LOGIN	TRADES	STATUS
U-001	Jane Doe	jane@example.com	Pro	Nov 28, 2025	Apr 03, 2026	87	ACTIVE
U-002	Carlos Mendes	carlos@tradelab.io	Elite	Oct 12, 2025	Apr 03, 2026	214	ACTIVE
U-003	Priya Nair	priya@hedgefund.ai	Elite	Sep 05, 2025	Apr 01, 2026	332	ACTIVE
U-004	Tom Brauer	tom@btcalgo.com	Pro	Dec 14, 2025	Mar 29, 2026	45	ACTIVE
U-005	Lena Fischer	lena@quant.de	Free	Jan 02, 2026	Mar 20, 2026	3	DISABLED
U-006	Marco Rossi	marco@pmfund.it	Pro	Feb 18, 2026	Apr 02, 2026	29	ACTIVE
U-007	Aiko Tanaka	aiko@algo.jp	Elite	Aug 30, 2025	Apr 03, 2026	501	ACTIVE
U-008	Dave Kim	dave@polybot.us	Free	Mar 10, 2026	Mar 15, 2026	0	DISABLED

Prototype Screens

11 – Admin

Disable user accounts

OBAnalyzer

ADMIN

← Menu

A

All Users

Disable Accounts

Reset Passwords

System Transactions

Platform Analysis

Toggle a user's account status below. Disabled users cannot sign in or access data until re-enabled.

ID	NAME	EMAIL	PLAN	STATUS	ACTION
U-001	Jane Doe	jane@example.com	Pro	ACTIVE	Disable
U-002	Carlos Mendes	carlos@tradelab.io	Elite	ACTIVE	Disable
U-003	Priya Nair	priya@hedgefund.ai	Elite	ACTIVE	Disable
U-004	Tom Brauer	tom@btcalgo.com	Pro	ACTIVE	Disable
U-005	Lena Fischer	lena@quant.de	Free	DISABLED	Enable
U-006	Marco Rossi	marco@pmfund.it	Pro	ACTIVE	Disable
U-007	Aiko Tanaka	aiko@algo.jp	Elite	ACTIVE	Disable
U-008	Dave Kim	dave@polybot.us	Free	DISABLED	Enable

Prototype Screens

12 – Admin

Reset user passwords

OBAnalyzer

ADMIN

← Menu

A

All Users

Disable Accounts

Reset Passwords

System Transactions

Platform Analysis

Send a password reset link to any user. The user will receive an email with a secure one-time link.

ID	NAME	EMAIL	STATUS	ACTION
U-001	Jane Doe	jane@example.com	ACTIVE	Send Reset
U-002	Carlos Mendes	carlos@tradelab.io	ACTIVE	Send Reset
U-003	Priya Nair	priya@hedgefund.ai	ACTIVE	Send Reset
U-004	Tom Brauer	tom@btcalgo.com	ACTIVE	Send Reset
U-005	Lena Fischer	lena@quant.de	DISABLED	Send Reset
U-006	Marco Rossi	marco@pmfund.it	ACTIVE	Send Reset
U-007	Aiko Tanaka	aiko@algo.jp	ACTIVE	Send Reset
U-008	Dave Kim	dave@polybot.us	DISABLED	Send Reset

Prototype Screens

13 – Admin

View
“system”
transact.

OBAnalyzerADMIN

← MenuA

All UsersDisable AccountsReset PasswordsSystem TransactionsPlatform Analysis

TOTAL TODAY8

REVENUE TODAY\$238

ERRORS1

WARNINGS1

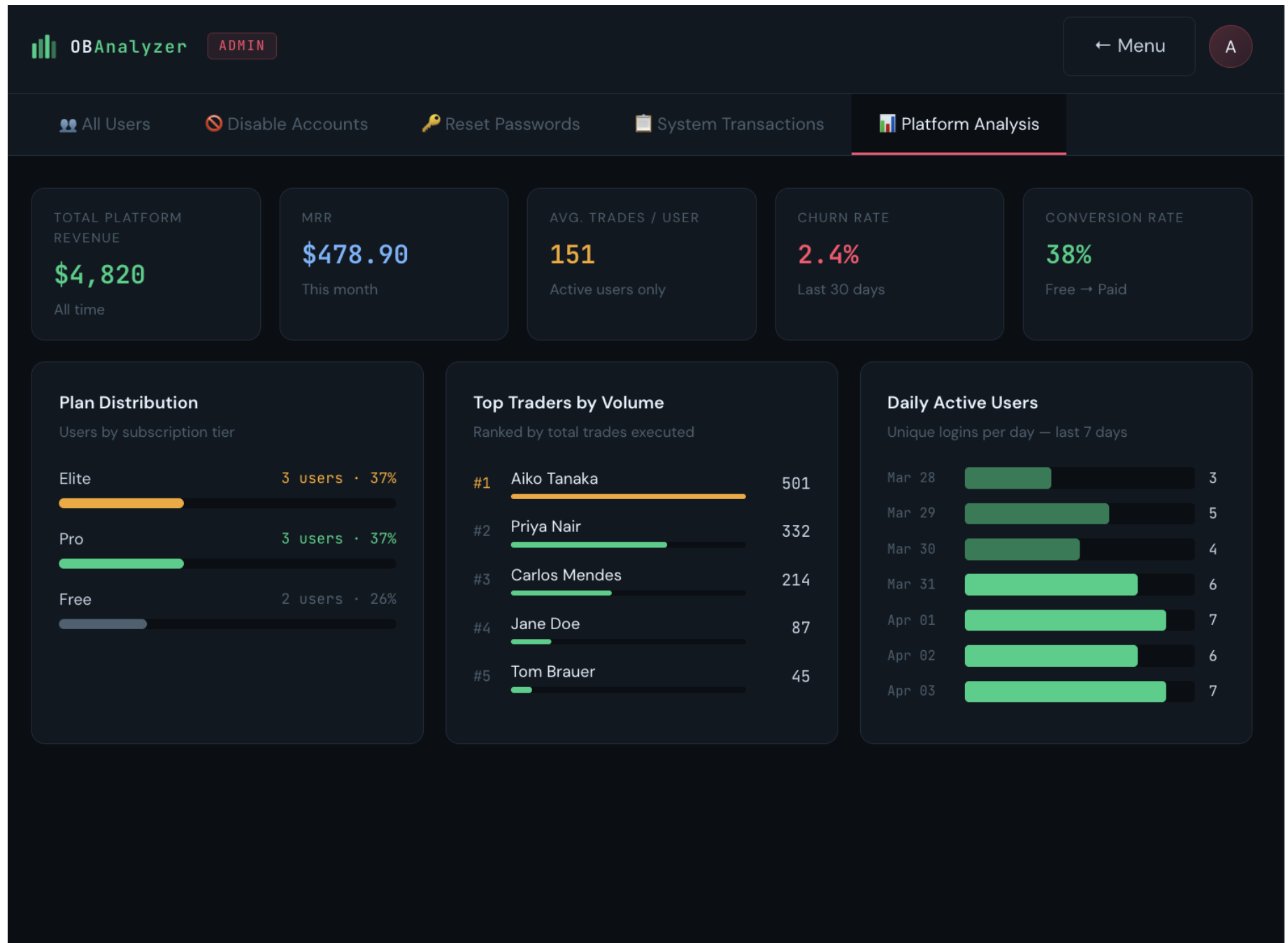
System Event Log

TXN ID	TIME	TYPE	USER	AMOUNT	STATUS
SYS-4421	11:04:02	Subscription Renewal	carlos@tradelab.io	\$29.99	SUCCESS
SYS-4420	10:58:31	New Registration	new_user@mail.com	—	SUCCESS
SYS-4419	10:47:14	Password Reset	tom@btcalgo.com	—	SUCCESS
SYS-4418	10:33:05	Plan Upgrade	priya@hedgefund.ai	\$99.00	SUCCESS
SYS-4417	10:21:49	Account Disabled	lena@quant.de	—	WARN
SYS-4416	10:10:02	Subscription Renewal	jane@example.com	\$9.99	SUCCESS
SYS-4415	09:58:37	Failed Payment	dave@polybot.us	\$9.99	ERROR
SYS-4414	09:44:11	Subscription Renewal	aiko@algo.jp	\$99.00	SUCCESS

Prototype Screens

14 – Admin

Perform analysis across all accounts



Updated Design

Architecture

Data source: Polymarket.

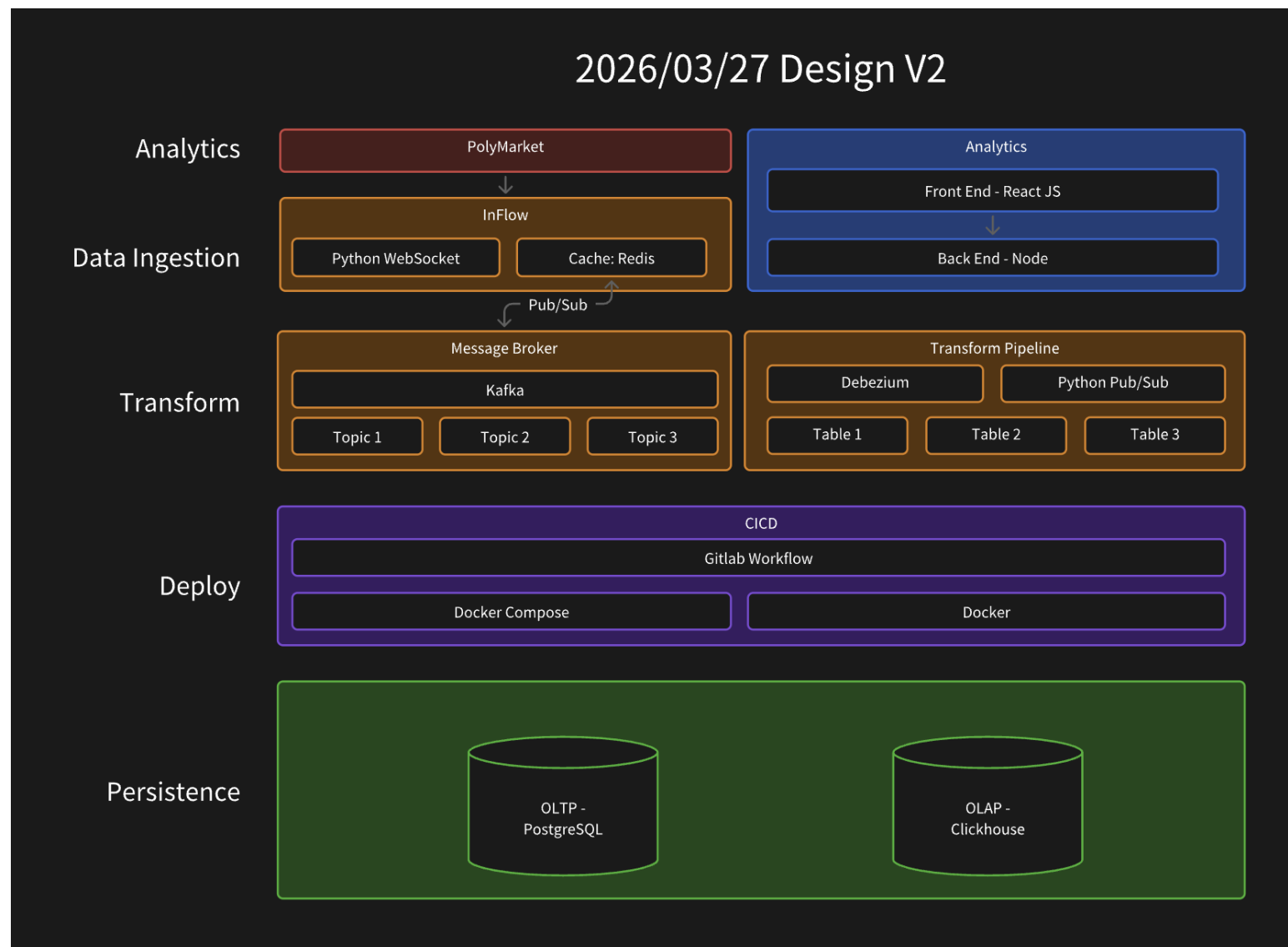
Ingestion layer: Python WebSocket collector with Redis cache.

Streaming backbone: Kafka message broker with multiple topics.

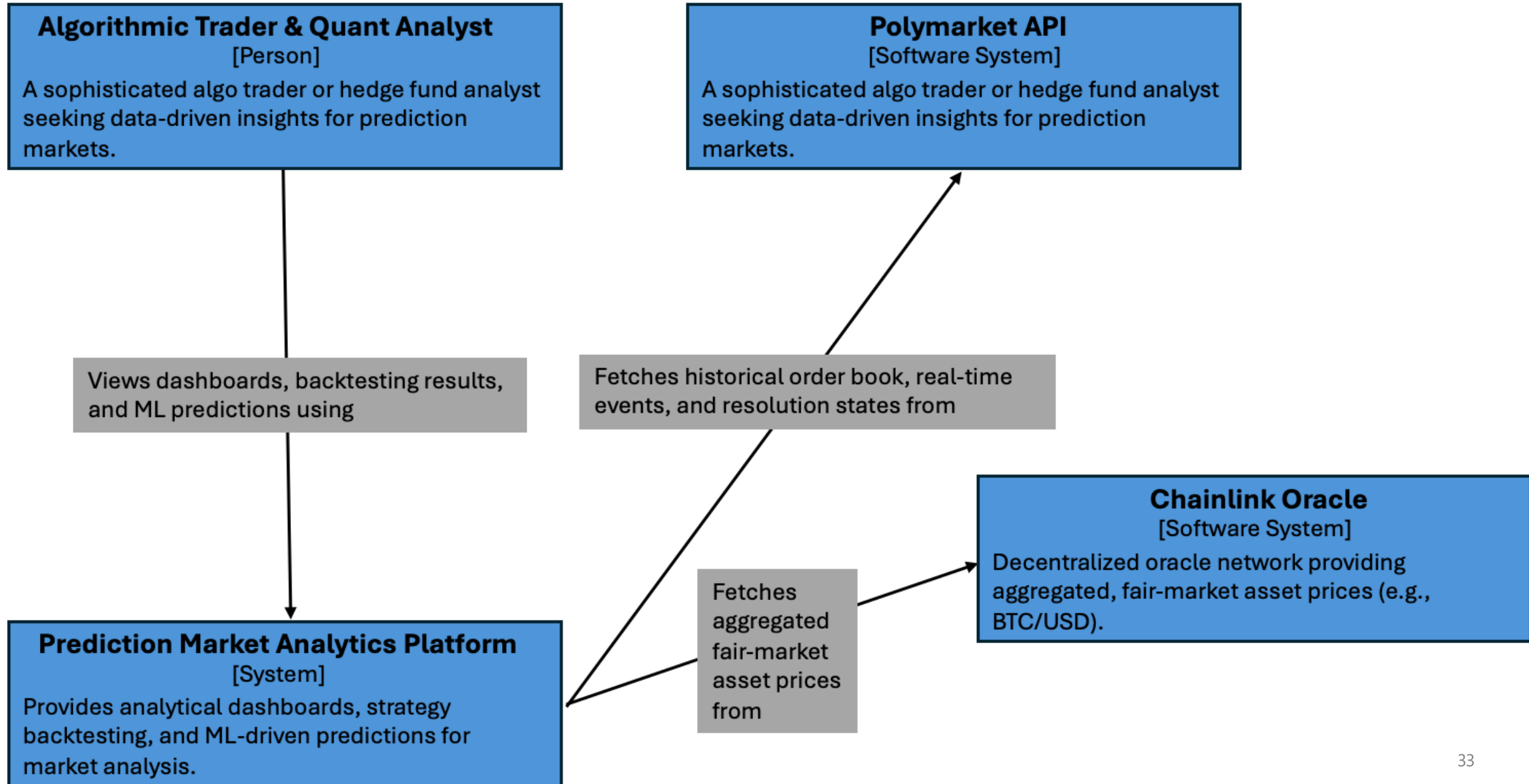
Storage: PostgreSQL (OLTP) for persisted normalized records.

Analytics backend: NodeJS services querying and aggregating stored data.

Analytics frontend: React UI for dashboards and research access.



System Architecture (C4 model) - Level 1 Context

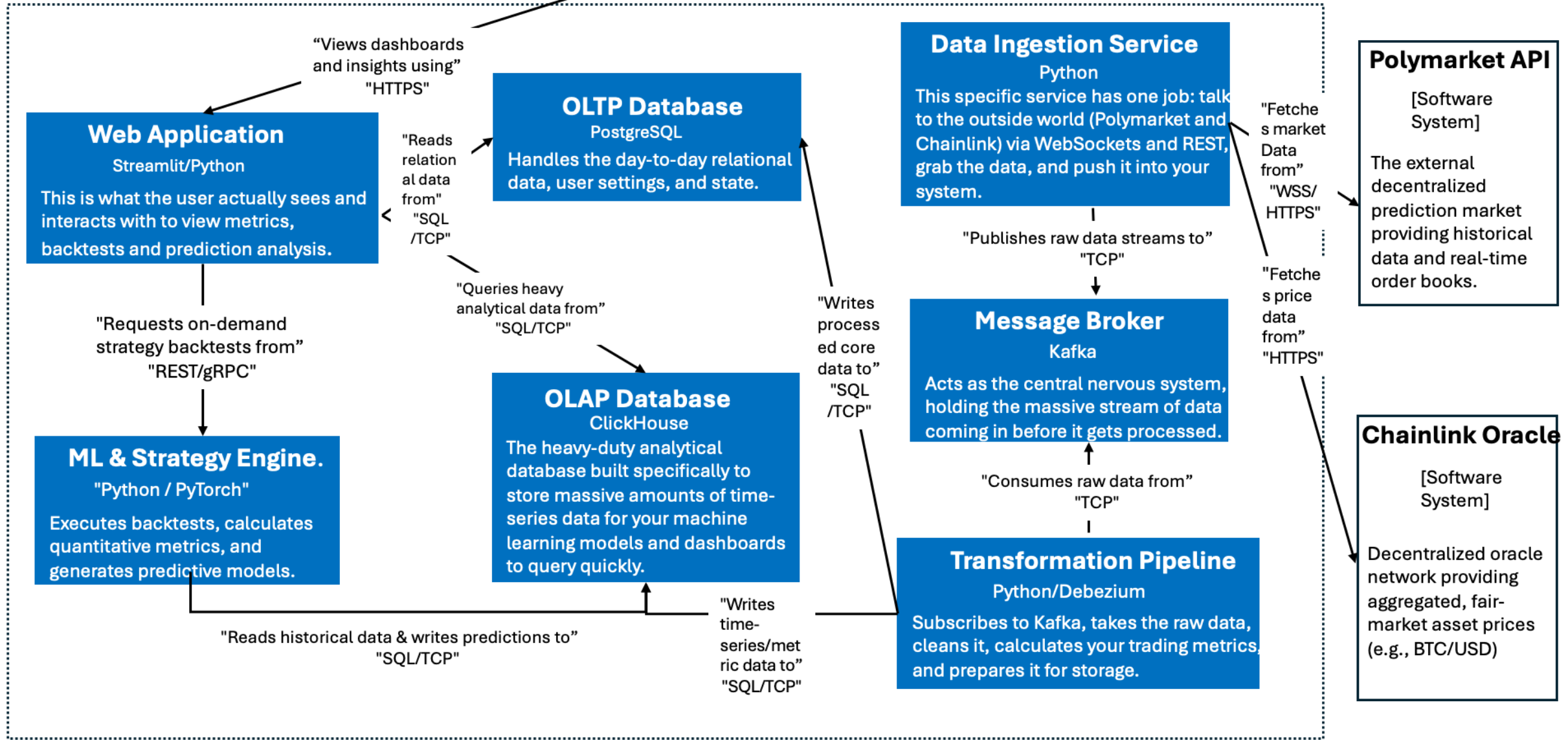


System Architecture (C4 model) - Level 2 Container

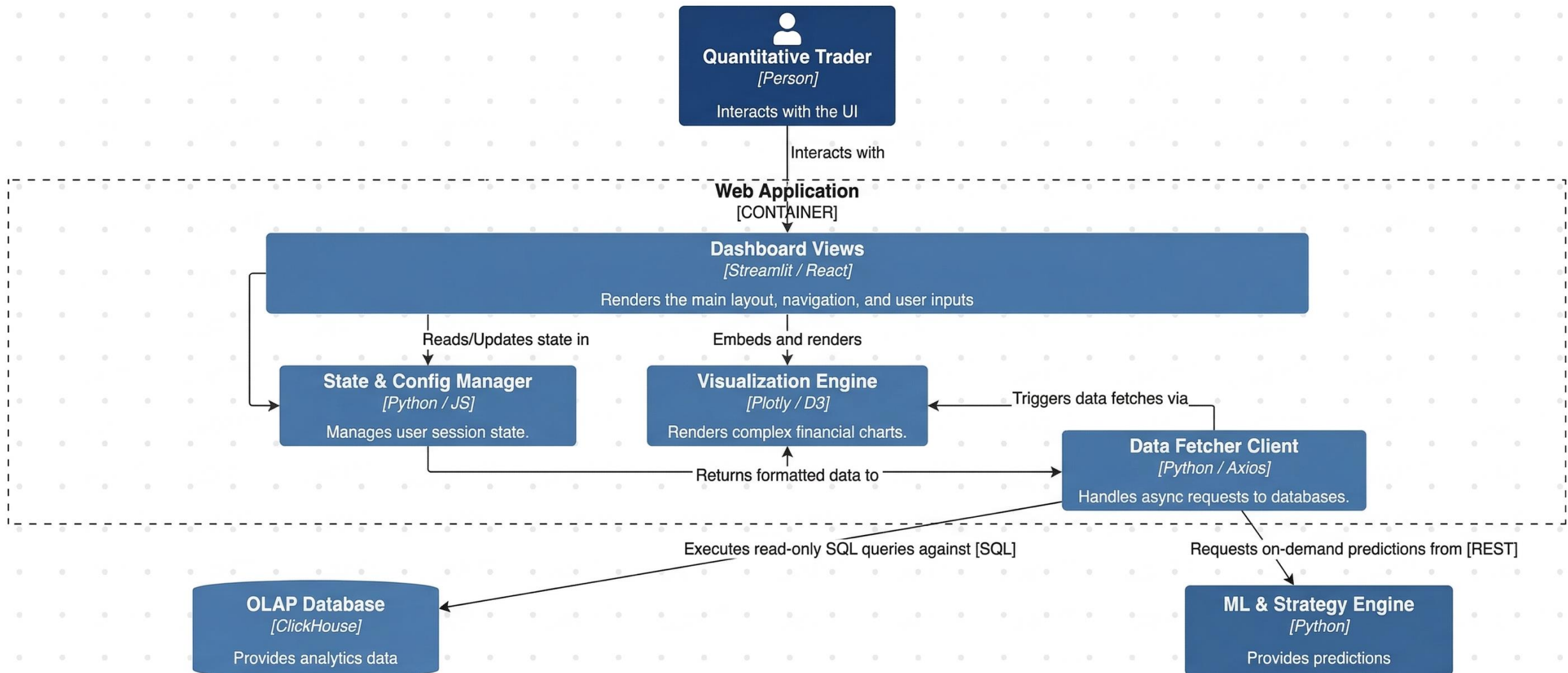
Algorithmic Trader & Quant Analyst

[Person]

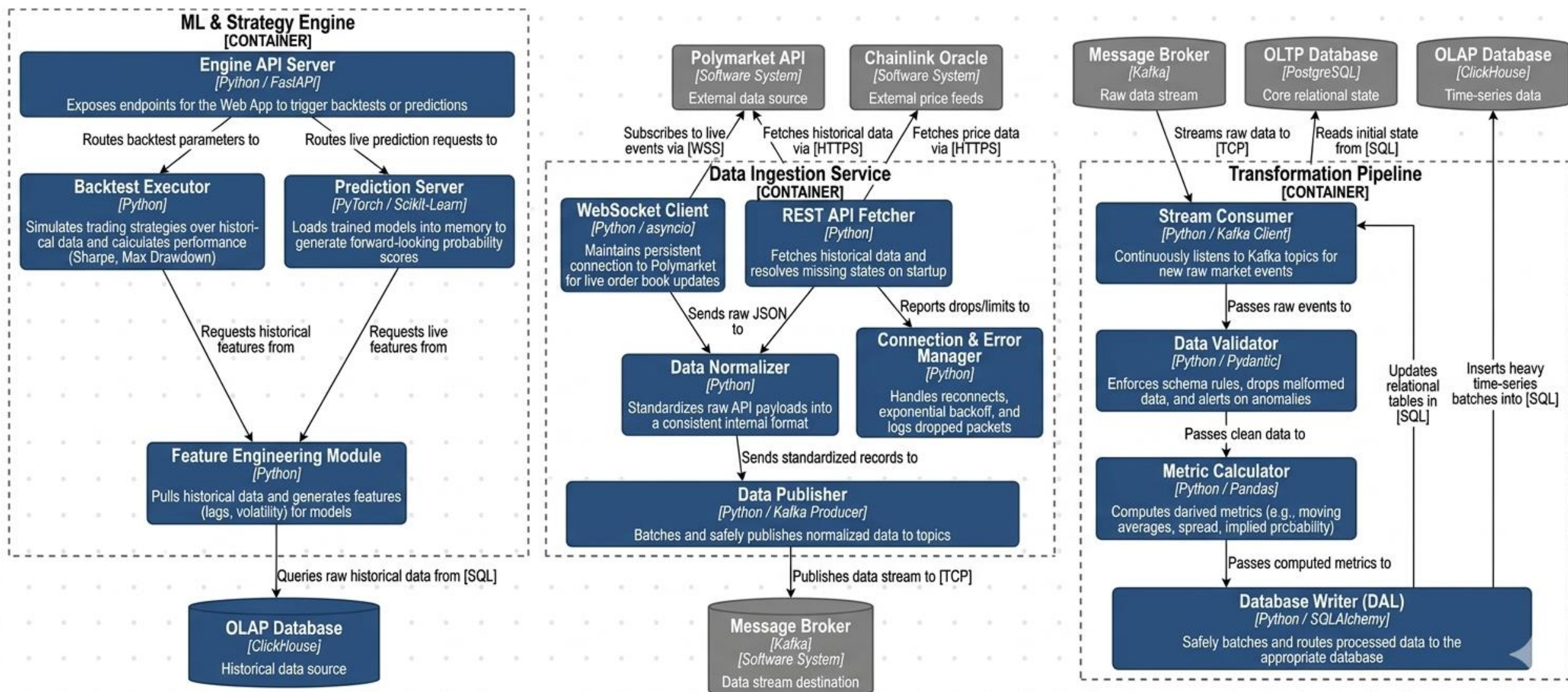
A sophisticated algo trader or hedge fund analyst seeking data-driven insights for prediction markets



Level 3 Component – Web Application



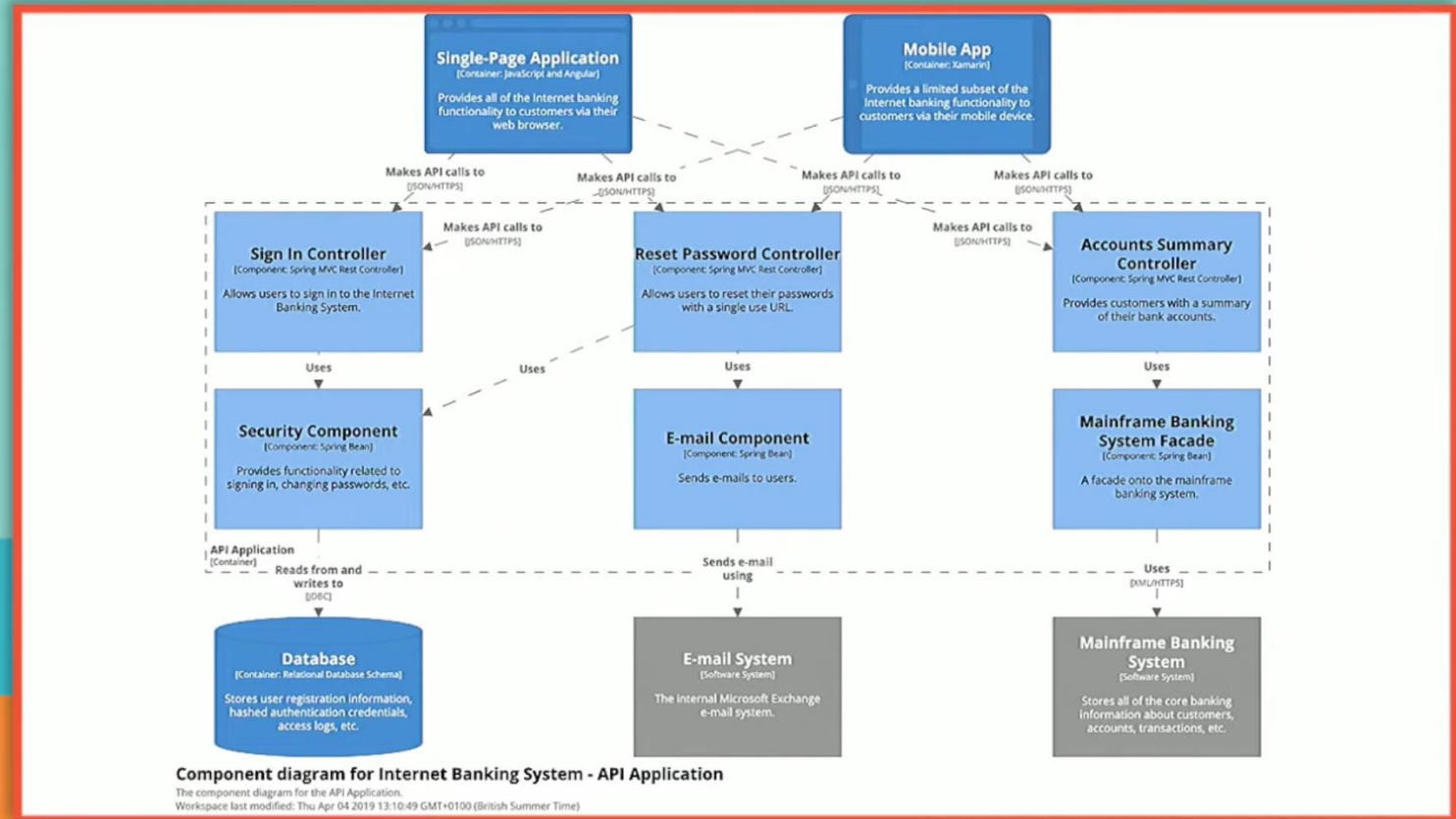
Level 3 Component – The Data Ingestion & Processing Containers



Level 4 Code



AGILE ON THE BEACH



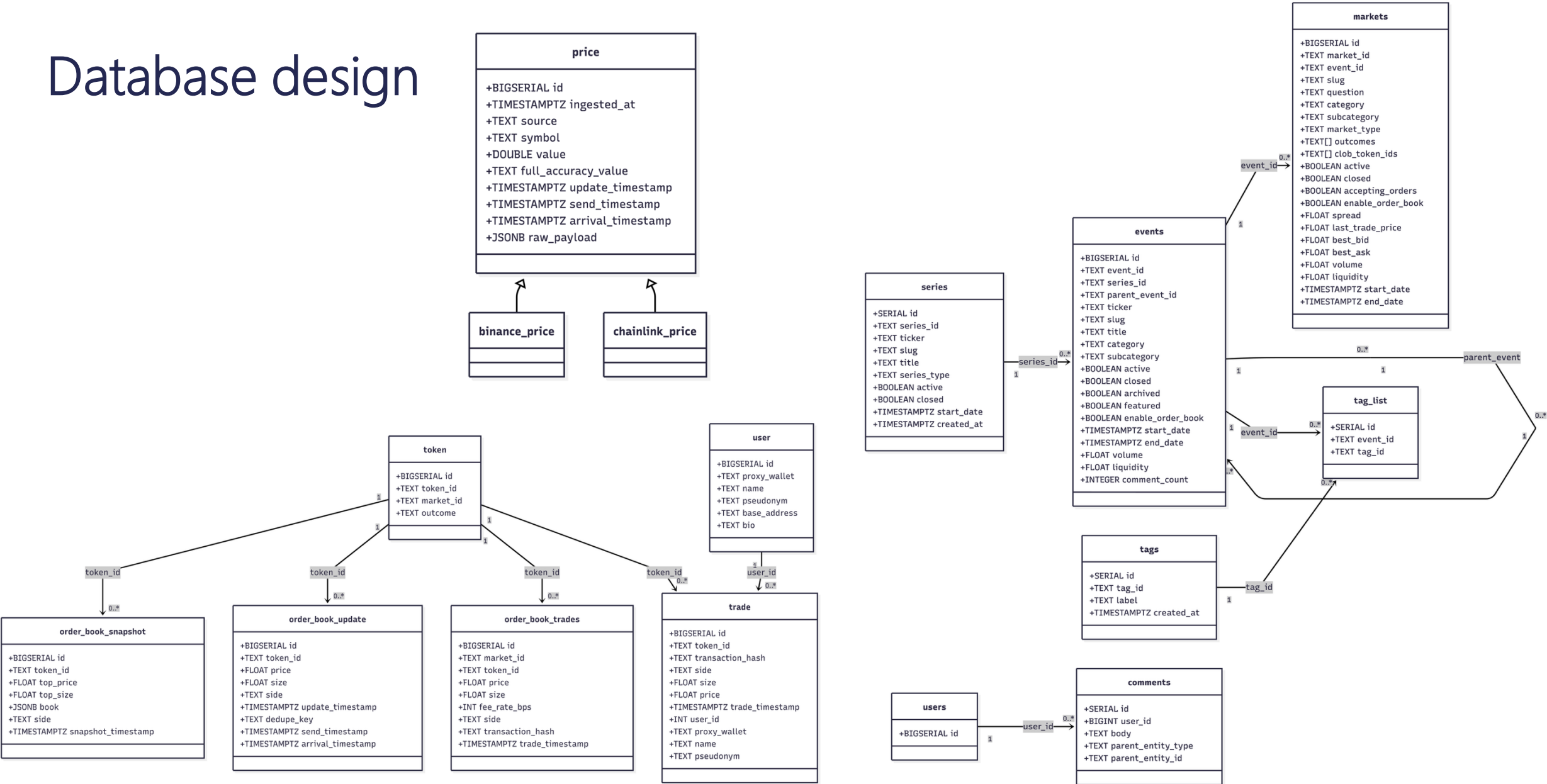
Streaming
sponsored by:

FALMOUTH
UNIVERSITY

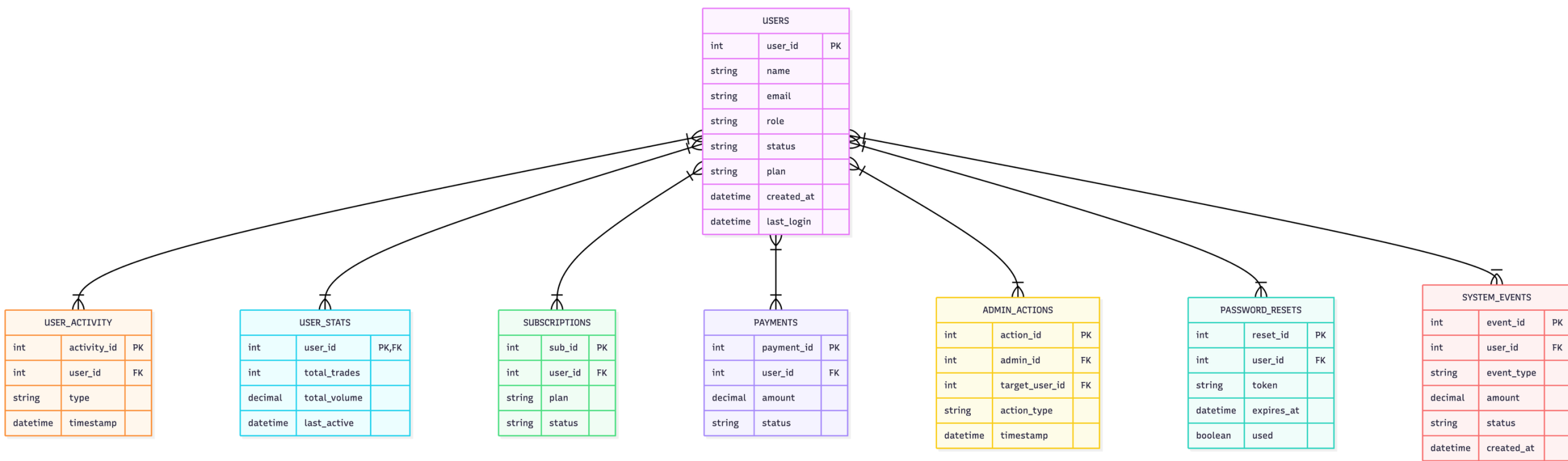
FLEXIBLE
LEARNING

always miss this I do not recommend
doing level 4 because it's not worth it

Database design



Database schema for users



Testing approach Methodology

Unit Tests (The Foundation):

Testing individual functions in isolation.

Task: Does this specific Python function correctly parse a Polymarket probability format into a float?

Integration Tests (The Middle):

Testing whether decoupled system modules communicate correctly.

Task: Does the WebSocket ingestion service correctly write data to the cache?
Does the message broker correctly ingest and forward the event?

Data Quality & Contract Tests (Pipeline Specific):

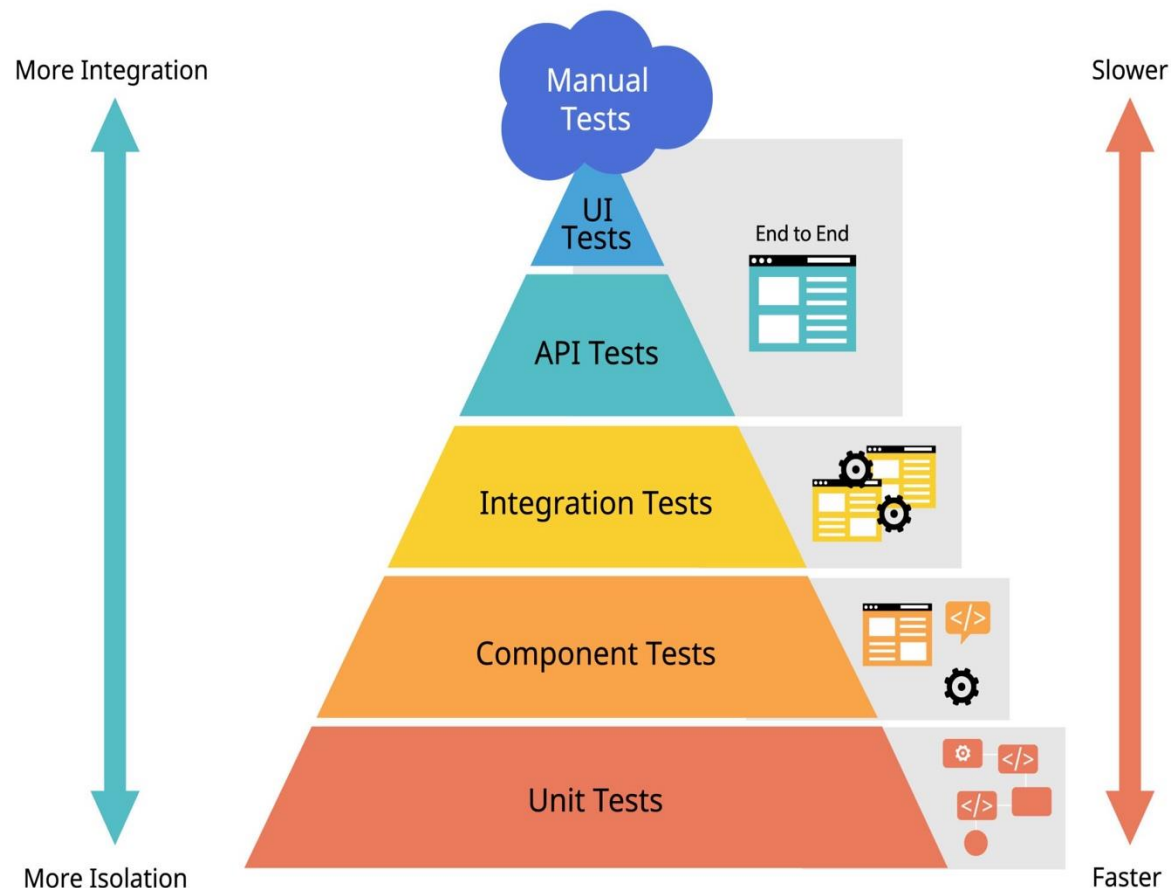
Testing whether schema or upstream data changes are handled safely.

Task: If Polymarket adds a new field to their JSON response, does our pipeline safely ignore it or crash?

End-to-End (E2E) Tests (The Peak):

Testing whether the full system works correctly from input to final user-visible output.

Task: If a trade event happens on the mock Polymarket WebSocket, does it successfully propagate through the pipeline, get stored in OLAP database, and appear correctly on the dashboard?



Testing approach In Data Ingestion

Corner Cases

Network Jitter & Disconnects.

What happens if the Polymarket WebSocket drops packets or silently disconnects? Does the system recognize the stale state?

Malformed Payloads.

Polymarket sends an unexpected null value for a betting pool size, or a string instead of an integer.

Redis Memory Eviction.

If the downstream pipeline slows down and Redis fills up, do we drop new data or block the WebSocket?

Examples from unit test to E2E test

Data Parsing

Objective: Verify that raw JSON from Polymarket is correctly transformed into our internal Python Data Class / Pydantic model.

Implementation: We do not need an internet connection for this. We save a sample Polymarket JSON response as a local fixture file. We write a pytest that feeds this static JSON into our parser function.

Python InFlow to Redis

Objective: Verify that parsed data is successfully cached in Redis with the correct Time-To-Live (TTL).

Implementation: We use Testcontainers (a framework that spins up a lightweight, throwaway Docker container of Redis just for the test). Our Python code connects to this temporary Redis instance, writes a mock message, and reads it back.

The Ingestion Slice

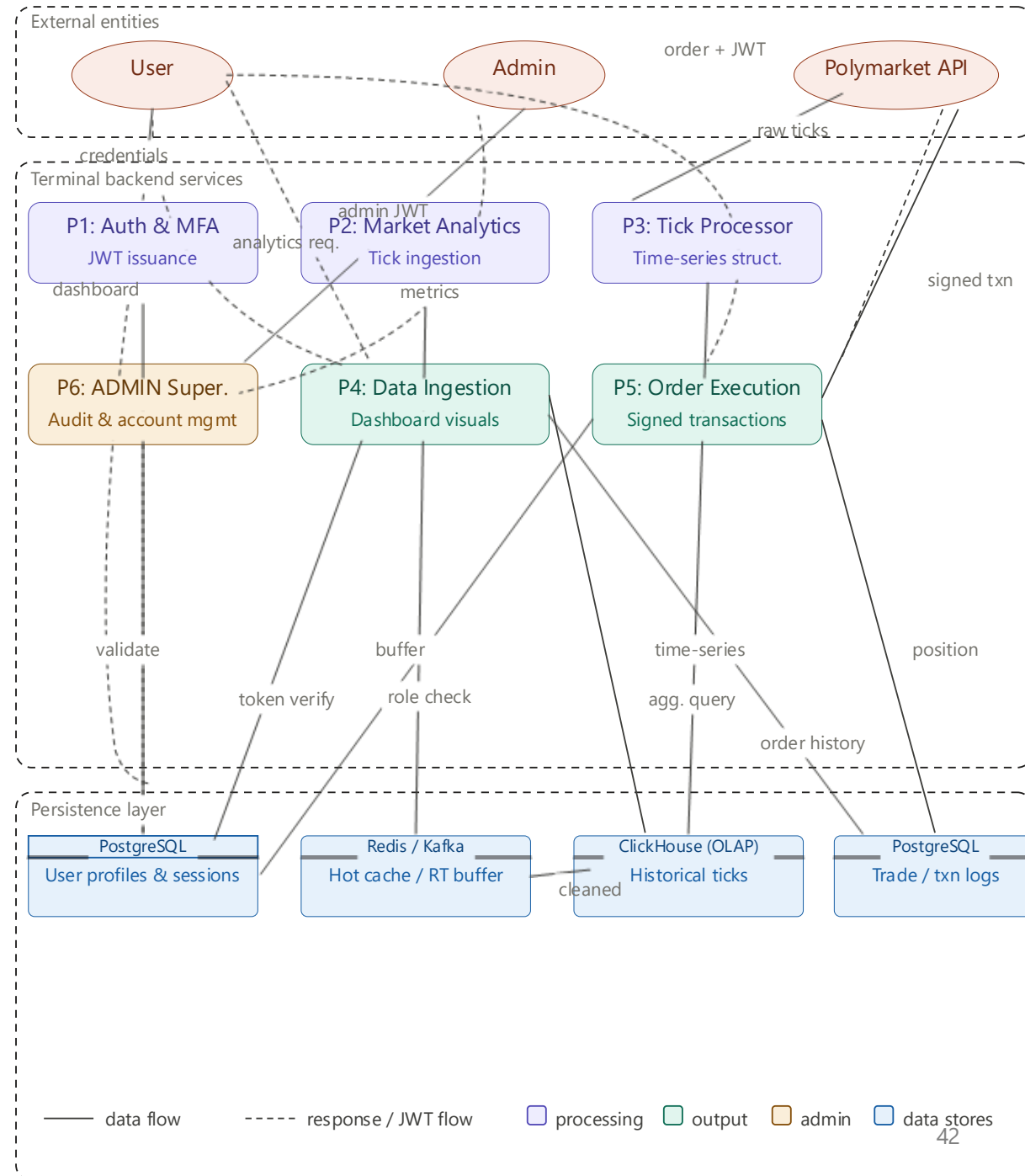
Objective: Verify the entire Inflow module works from WebSocket to Message Broker.

Implementation: We create a mock WebSocket server that mimics Polymarket and emits pre-determined trade events at a specific rate. We spin up our Python InFlow service, a Redis instance, and a single Kafka node using Docker Compose.

Data Flow Diagram (DFD)

System Overview:

- Users interact with the system through frontend connected to backend APIs
 - Backend services process authentication, logics and provide database access
 - External market data – Polymarket is ingested, processed and stored
 - Data views as dashboards and analytics components.
-
- **Assets:** User credentials and Oauth tokens, user permissions, Orderbook data and trade records, market structure data, internal APIs and service endpoints, PostgreSQL database contents, external market data (Polymarket based), system logs, audit records.
 - **Actors:** User, Administrator, Polymarket API, Backend Services (for handling logics and data processing)
 - **Threats: Based on STRIDE analysis**



Data Flow Diagram (DFD) – STRIDE analysis

All S T R I D E

CATEGORY	SEV.	NODES	THREAT	MITIGATION
Spoofing	H	User P1	JWT/session token stolen & replayed	Short-lived tokens + revocation list in D1
Spoofing	H	PoLyAPI P2	Polymarket API spoofed, fake tick data injected	TLS cert pinning + HMAC signature on every feed
Tampering	H	P5 PoLyAPI	Order request tampered before signing	User-signed payload; P5 verifies hash against receipt
Tampering	H	P2 D2	Tick data corrupted in Redis/Kafka buffer	Per-message checksum; P3 validates before writing D3
Repudiation	H	P5 D4	User denies placing order — no audit trail	Append-only signed order log in D4
Repudiation	M	P6 D1	Admin denies executing command	Immutable admin audit log; ship to off-system SIEM
Info disclosure	H	D1 P1	User PII leaked via errors or SQL injection	Parameterised queries; encrypt PII columns; no stack traces
Info disclosure	H	P6 PoLyAPI	API keys exposed via overexposed admin endpoint	P6 behind VPN/allowlist; secrets manager for all keys
Denial of service	H	P2 D2 P3	Tick flood overwhelms data pipeline	Rate-limit at P2; back-pressure on Kafka; circuit breaker
Denial of service	M	P4 D3	Expensive analytics queries degrade OLAP store	Per-user query budget + result caching in P4
Elevation of priv.	H	P6 P4 P5	Regular user JWT accepted on admin P6 endpoints	Deny-by-default middleware; role validated on every request
Elevation of priv.	H	P5 D1	JWT role claim tampered to bypass order checks	RS256 signing; re-fetch authoritative role from D1

STRIDE analysis:

- Spoofing
 - Stolen session or OAuth token, external data source is not true.
- Tampering
 - Modified API requests, data corruption and wrong market data.
- Repudiation
 - Users/admin deny actions, missing logs for audit.
- Information disclosure:
 - Data leakage like user information and API keys, and overexposed admin endpoints.
- Denial of service
 - Data pipeline overload.
- Elevation of privilege:
 - Users access admins endpoints, and lack of authorization check.

Data Flow Diagram (DFD) - Team Discussion

'What are we working on?'

Can we see where all the data goes and who uses it?

- Yes, we can see all the data goes and the actors by nodes connections, and comments between those node represent the relations.
- The entities interact with which endpoints are clear.

Do we see the processes that move data from one data store to another?

- Yes, the diagram shows exactly how backend processes data in transit.
- For example, the Tick processor (P3) taking buffered data from Redis/Kafka and moves TS structure to Clickhouse OLAP store.

Does the diagram show all the trust boundaries, such as where different accounts interact?

- Yes, the bounding box separates boundaries by "External Entities", "Terminal backend Services", and "Persistence Layer".

Can we look at the diagram and see exactly where the software will make a security decision?

- Yes, a security decision will be made by Auth & MFA (P1), and Admin Super (P6) as authentication decisions and authorized role checking processes run.

Does the diagram reflect the current or planned reality of the software?

- Yes, the software is well-defined by DFD.
- Including specific database including PostgreSQL, ClickHouse, Redit, and external API (Polymarket).

'What can go wrong?'

Have we looked at each element of the diagram?

- Yes, we have fully reviewed the API gateways, the internal backend processes from P1 to P6 and the data storages.

Have we looked for each of the STRIDE threats?

- Yes, we have fully reviewed each threat by STRIDE analysis, by that two vulnerabilities for every single category are being checked.

For each threat, can we describe the actor, the attack, and the impact?

- Yes, we table has defined the nodes involved, with severity (High/Median), and the nature of attack such as stealing a JWT token or attacker flooding within the pipeline.

'What are we going to do about it?'

Is there a proposed/planned/implemented mitigation to address each threat?

- Yes, we have mitigations for each threat based on STRIDE and also included in the table.

Are there test cases we should write?

- Testing our API endpoints for SQL injection vulnerabilities.
- Simulating tick data floods to ensure P2 rate, which limiting and Kafka back-pressure hold up.
- Attempting to access P6 administration endpoints with regular user JWTs to verify our deny-by-default middleware.

Testing approach - Unit test suite for Full Stack

Corner Cases

Invalid User Inputs. (frontend validation)

A user submits invalid format of forms, registration info, or empty data. The system needs to prevent submission and provide clear validation feedback in the UI level.

Unauthorized Access. (security)

A user tries to access admin routes or attempt to get access to API keys. Access control should timely focus on across both frontend and backend (render and authorization).

API Failure, Timeout or Error Response.

The backend may return an error or slow responses during user requests. The frontend should handle those scenarios without crashing or exposing internal errors.

Invalid Request, Malformed Payload (backend validation)

Users' requests may contain missing fields or incorrect data types or values. The backend should validate the inputs and reject those request correctly.

Examples

Frontend Component Validation

Objective: Verify that the UI components correctly validate the user input.

Implementation: We render components (login/registration) using testing libraries, simulate possible user inputs (forms/types), and verify validation behavior (without real APIs); thus, the test remain isolated at the component level.

API Helper Function (Frontend)

Objective: Verify that the frontend API utilities construct and handle the requests correctly from backend without too much lagging

Implementation: We monitor backend responses without calling a live server, and test the formats of requests, success handling and error states. We then verify the successful responses or error messages are correctly shown.

Backend Authorization

Objective: Verify that the protected routes enforce role-based access controls.

Implementation: We simulate authentication tokens and user roles, then test if the endpoints returning the responses correctly. Such as 401 Unauthorized for missing or invalid authentication and 403 Forbidden for insufficient privileges.

Backend Logic with interaction of Database

Objective: Verify if the logic independently of the database, including validating request payloads, transforming input data, returning expected outputs and failure case handling.

Implementation: We call on the database and test if the controller or service functions ensures the correct outputs and the error handling. For example, we simulate correct reads/writes, missing records or database exceptions.

Testing approach - Unit test suite for Data Service

For Inflow Gateway, we use a strong mocking setup for unit tests.

- pytest monkeypatch
- custom test doubles
- fake WebSocket connections
- recording publishers
- fake HTTP responses

On the right are a few corner cases, and how we test for them.

1. External dependency failures

- WebSocket API unavailable
- Redis unavailable
- network I/O interruption

2. Data processing edge cases

- incorrect or unexpected data transformation
- invalid market filtering
- missing or incomplete payload enrichment

3. Resilience and recovery scenarios

- timeout during data fetch
- WebSocket connection failure
- retry behavior after failure
- graceful degradation when a dependency breaks

4. Data quality and integrity issues

- duplicate message detection
- malformed or partial payloads
- incorrect publish behavior

APIs

External Polymarket Gamma REST endpoints documented:

- `GET /events`
- `GET /series`
- `GET /trades`

Internal data retrieval endpoint:

These endpoints retrieve data from ClickHouse and expose it for backend consumption

- `POST /internal/clickhouse/write`

Allowed target tables:

- `chainlink\_prices`
- `binance\_prices`
- `btc\_contracts\_ob\_vol`
- `btc\_contracts\_ob\_midprice`

Please refer to the `api.yaml` file in the repository for full API details.

Data Service API 0.1.0 OAS 3.1

/openapi.json

chainlink

GET

/chainlink/prices Get Chainlink Prices

health

GET

/health Health

Schemas

HTTPValidationError > Expand all **object**

ValidationError > Expand all **object**

```
{
  "symbol": "BTC/USD",
  "start": null,
  "end": null,
  "count": 1000,
  "data": [
    {
      "update_timestamp": "2026-04-01T23:47:46Z",
      "value": 68119.13302705284,
      "symbol": "btc/usd"
    },
    {
      "update_timestamp": "2026-04-01T23:47:47Z",
      "value": 68119.13302705284,
      "symbol": "btc/usd"
    },
    {
      "update_timestamp": "2026-04-01T23:47:48Z",
      "value": 68121.44061556352,
      "symbol": "btc/usd"
    },
    {
      "update_timestamp": "2026-04-01T23:47:49Z",
      "value": 68125.2403528811,
      "symbol": "btc/usd"
    }
  ]
}
```

Response headers

```
content-length: 85176
content-type: application/json
date: Fri, 03 Apr 2026 18:53:13 GMT
server: uvicorn
```

Error handling

Domain & Logic Safeguards (Task-Driven)

Malformed Ingestion Payload (Schema Changes)

Mechanism: Dead Letter Queue (DLQ). Catch `ValidationError`, route bad data to a separate Kafka topic, and skip to the next message without crashing the pipeline.

Quantitative Calculation Exceptions (e.g., Missing Data, `ZeroDivisionError`):

Mechanism: Graceful Degradation. Wrap core math in try-except blocks. Output `stale_data` flags or safe previous values to prevent NaN or Infinity from polluting the Clickhouse OLAP.

Infrastructure & System Safeguard (NFR-Driven)

Downstream Service Timeout (>500ms API/DB response)

Mechanism: Circuit Breaker Pattern. Temporarily halt requests to the failing service to prevent cascading failures; serve historical fallback data from the Redis cache.

Kafka/WebSocket Connection Drops

Mechanism: Exponential Backoff with Jitter. Catch `ConnectionError` and reconnect with increasing, randomized delays (e.g., 1s, 2s, 4s + random ms) to avoid network storming.

Spike in Trade Volume (Consumer Lag)

Mechanism: Backpressure & Auto-Scaling. Monitor Kafka consumer lag continuously. If thresholds are breached, trigger AWS ECS to dynamically spin up additional Python processing containers.

Non-Functional Requirements

Real-Time Performance

Low-latency ingestion and processing for live Polymarket data, enabling timely analytics and strategy response.

Reliability & Fault Tolerance

Automatic retry, reconnection, buffering, and graceful degradation to keep the pipeline running under partial failures.

Scalability

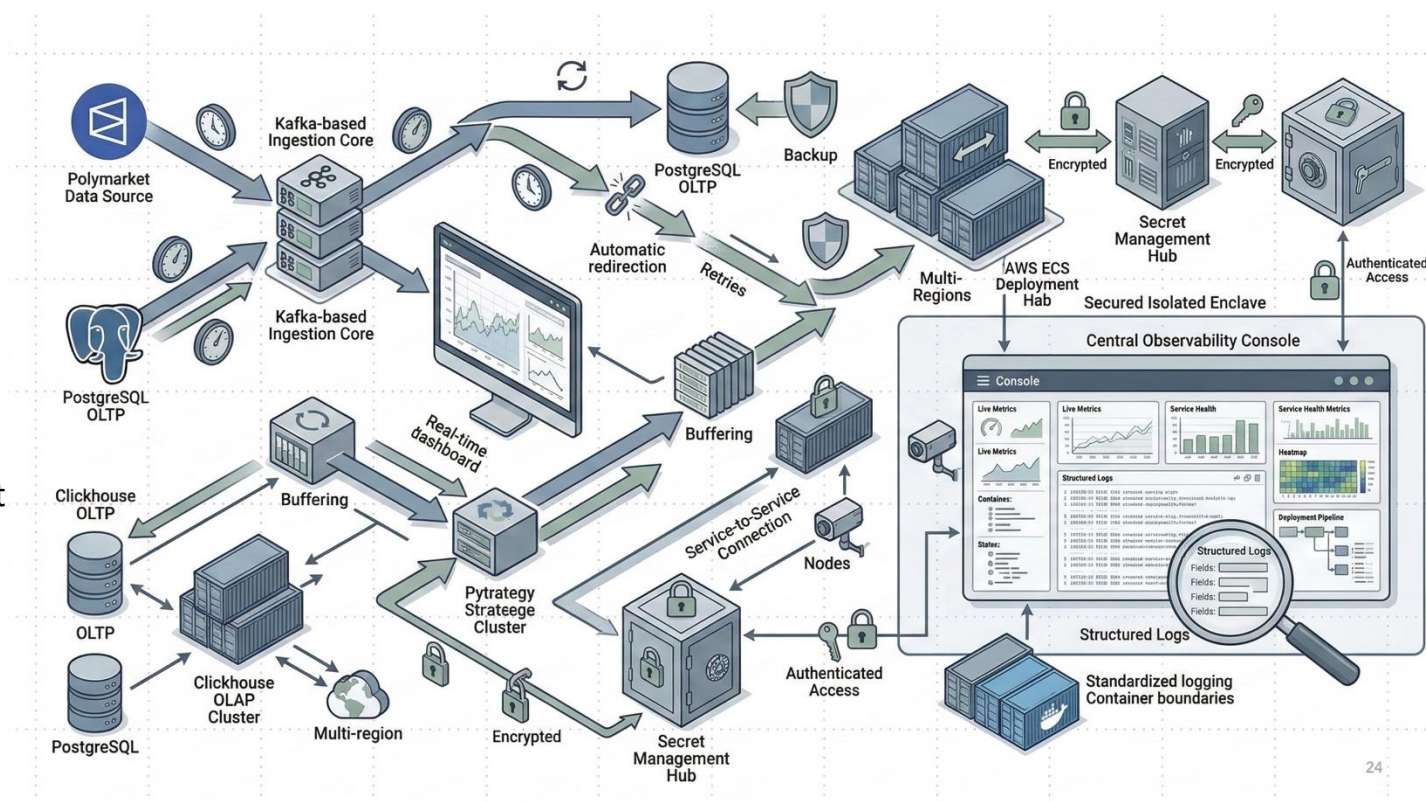
Modular services and message-driven architecture allow independent horizontal scaling as market volume grows.

Security & Access Control

Secure secret management, authenticated service access, and environment isolation to protect system resources and credentials.

Maintainability & Observability

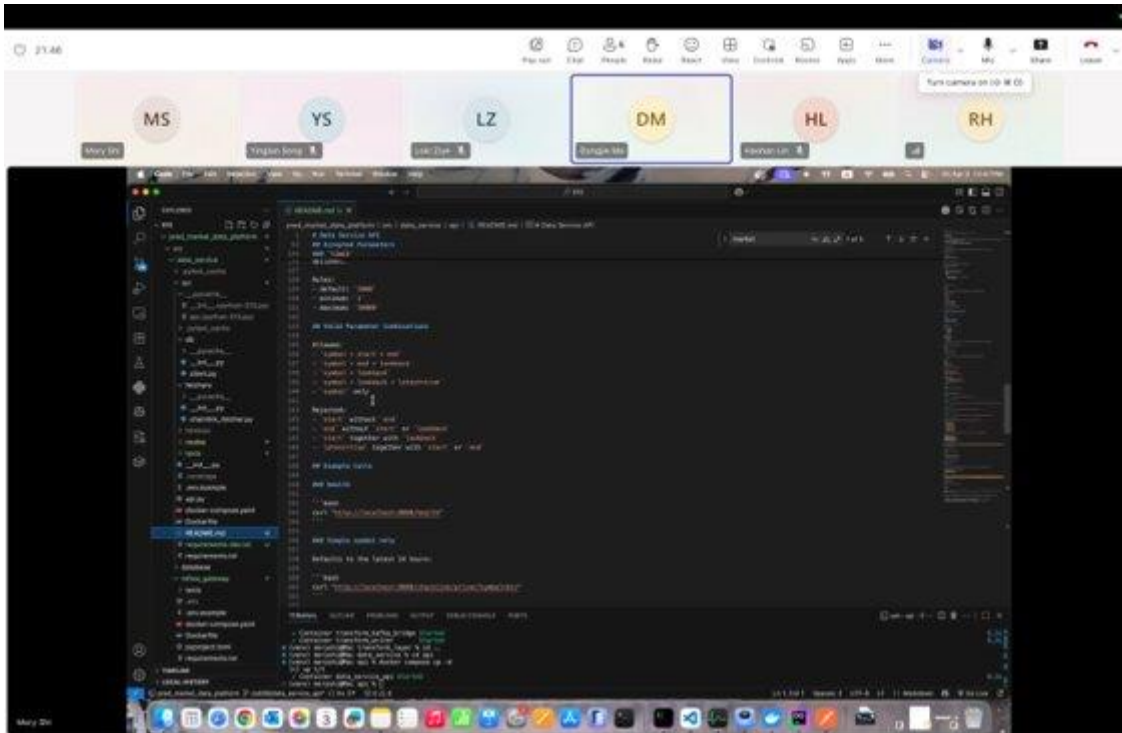
Clear module boundaries, containerized deployment, structured logging, and monitoring to simplify debugging, testing, and future extension.



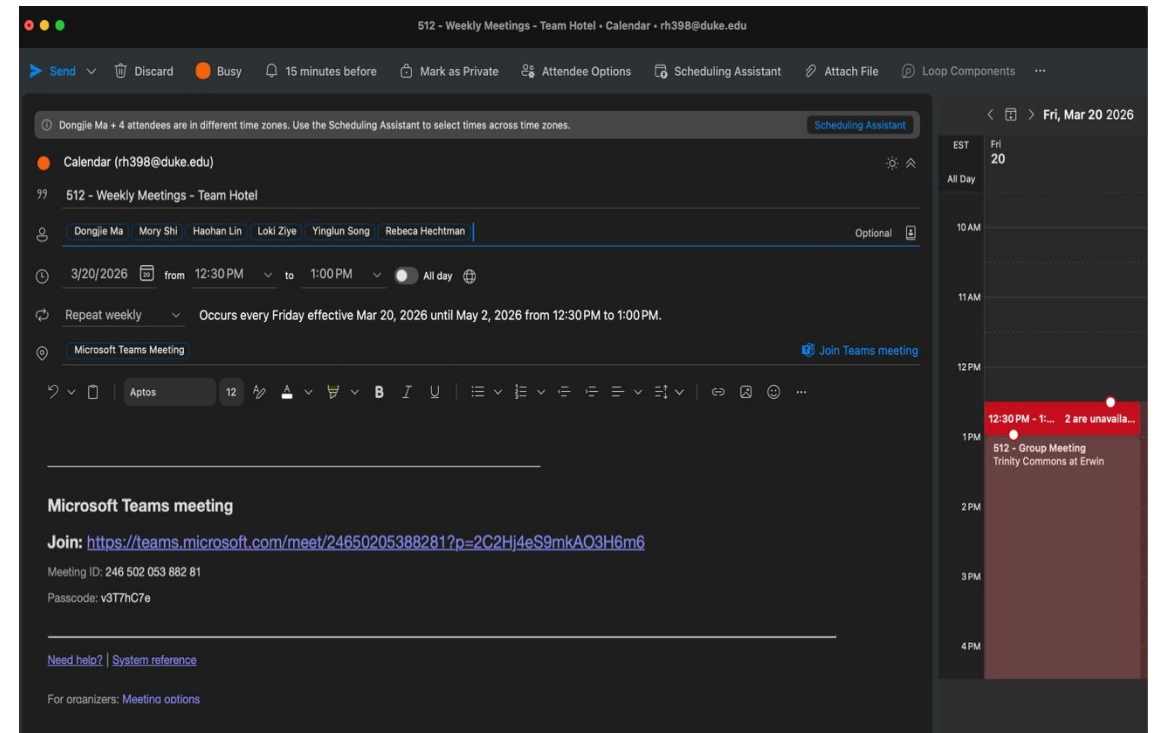
Status, Review and Planning

TA Discussion and Weekly Meetings Scheduled

We successfully met with our assigned TA today (April 3rd) to review our project progress and next steps



Next meeting with Dongjie Ma on April 10 (weekly meetings on Friday at 12:30)



Sprint Review Summary

Design & Prototyping: In Week 4, the team continued refining the full stack design by moving from ideation into implementation. Progress was made on CSS styling, HTML page development, and JavaScript-based user interaction for the main user interface prototypes. Admin functionality was also further detailed, with tab designs covering user management, password reset, and system-wide analysis features.

Architecture & Analysis: The project architecture became more concrete this week as the team advanced the data service design and its role within the broader full stack system. Testing plans were further developed, and the transformation workflow and DFD-related work were refined to better support system integration and future validation.

Backend & Integration: A major Week 4 milestone was the API work connecting the full stack application with the data service. Progress was also made on linking the front end to the API through JavaScript, while database implementation for users in PostgreSQL and update transformation tasks moved forward. OAuth2 authentication integration was initiated, helping establish the basis for secure user access and backend connectivity.

Processes & Collaboration: Team coordination remained active through continued GitLab issue updates and discussions around development workflow. Week 4 also marked the start of planning for CI/CD, containerization, and security analysis tasks, ensuring that upcoming development will be supported by stronger deployment, testing, and monitoring practices.

Sprint Retrospective Summary

Looking back at Sprint 4, the team successfully collaborated to advance our full-stack and data services. We divided our efforts between frontend and backend development and conducted preliminary API testing. Additionally, we made significant progress in refining our data flow diagrams and constructing a clearer STRIDE analysis to ensure system security.

On the technical side, we completed the frontend page design, backend architecture, and database implementation. We also finalized the API design for the backend data services and completed the initial round of testing.

For Sprint 5, our primary focus will be on integrating the various service endpoints and conducting an end-to-end trial run of the system. Furthermore, we will evaluate our current progress and timeline to determine whether to initiate development on the administrator module.

Sprint Planning Summary

For Sprint 4, our objective shifts from active development and implementation into the highest-priority items currently in our project backlog. This process did not go without challenges:

1. Still experiencing difficulties integrating the front end, back end, and data layer APIs.
2. Have a continuous and stable flow of data
3. Share the VCM

We were able to overcome some of these challenges and keep working on the rest

And for next week, sprint 5, we plan to tackle the finalize the initial implementation and highest priority structure which we designed this week. Key deliverables for the upcoming week include

1. Finalizing the user and admin database,
2. Integrate APIs and create continuous and stable data flow
3. Polishing the web design and back-end design
4. Containerize functions for efficiency

We continuous to have our recurring weekly TA meeting for Friday 12:30 to ensure we are catching the progress. And we have schedules more group meetings next few weeks to make sure the perfection of our final presentation.

Additional Comments and/or Concerns

General Comments: We are at the stage of deploying our project to the virtual machine. We need to make sure the CI/CD flows well, at the same time we are trying to make sure that every week we are doing what we need to do so that we can finish the project on time.

Risks/Blockers: Currently, there are no major blockers. However, we will be actively monitoring concerns as the ones below:

Risk1 – Operational Overhead: Our approach strongly resembles the microservice architecture, while it allows for better modulization, it also adds maintenance overhead to the team. We need to closely monitor the status of each service, and also added more automated orchestrations such as CI/CD flows to ensure smooth operations.

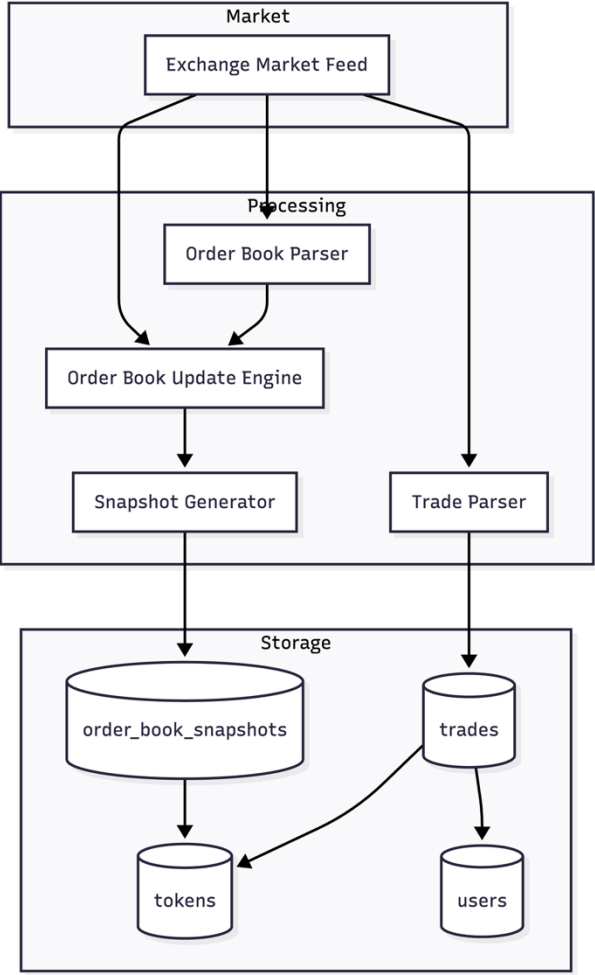
Risk2 – API Access Concern: Polymarket may retrieve/change the accessibility to their official API endpoints, and it will significantly impede our development.

Risk 3 – Virtual Machine Deployment Needed: We need to deploy the project onto machine so that we know each component works smoothly with each other.

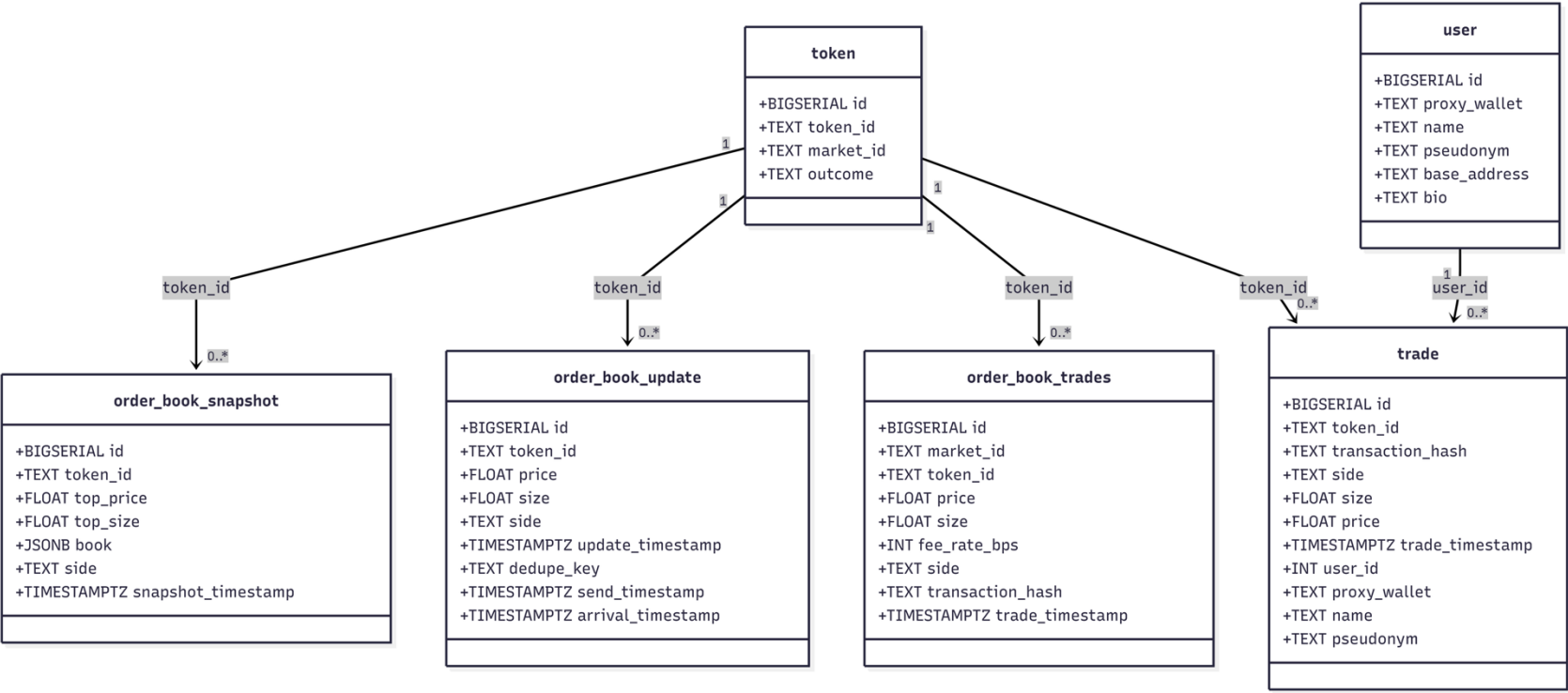
Risk 4 – Massive Data Storage Need: As we are capturing high frequency tick-level orderbook data, the need for storage is overwhelming. We are looking at the need of roughly ~20 GB of storage need per day.

Annex (initial documents created)

UML Class Diagram – Order Book

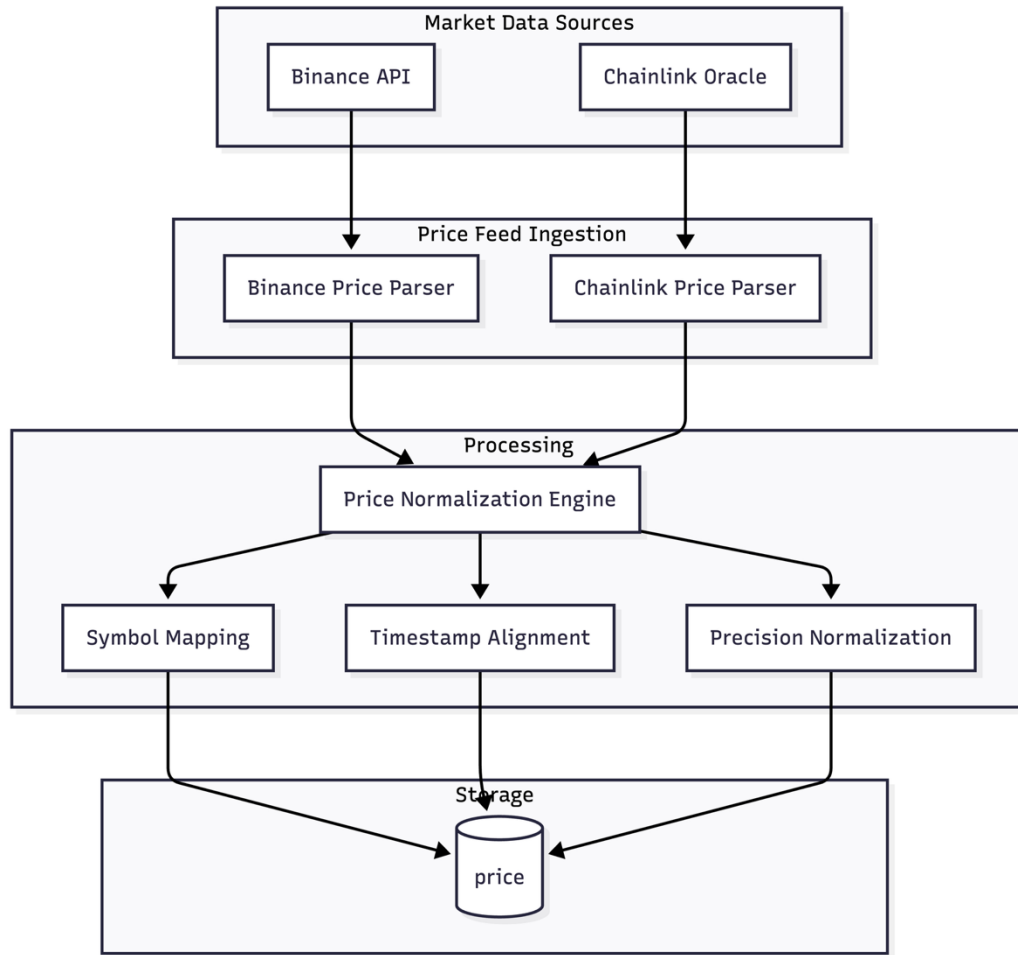


Architecture/pipeline

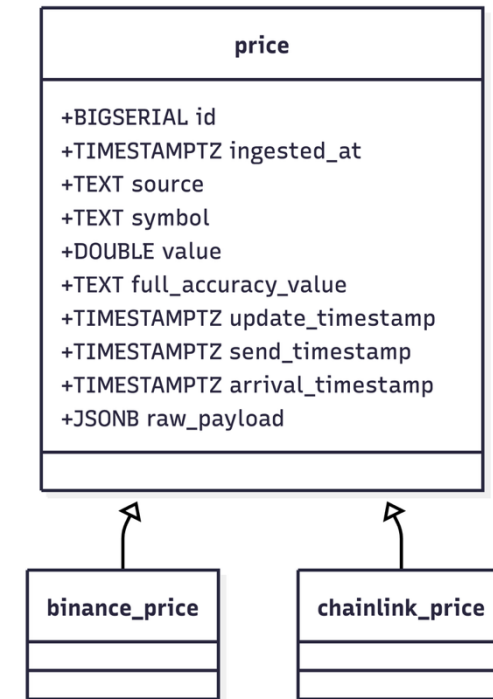


UML Class Diagram

UML Class Diagram - Price

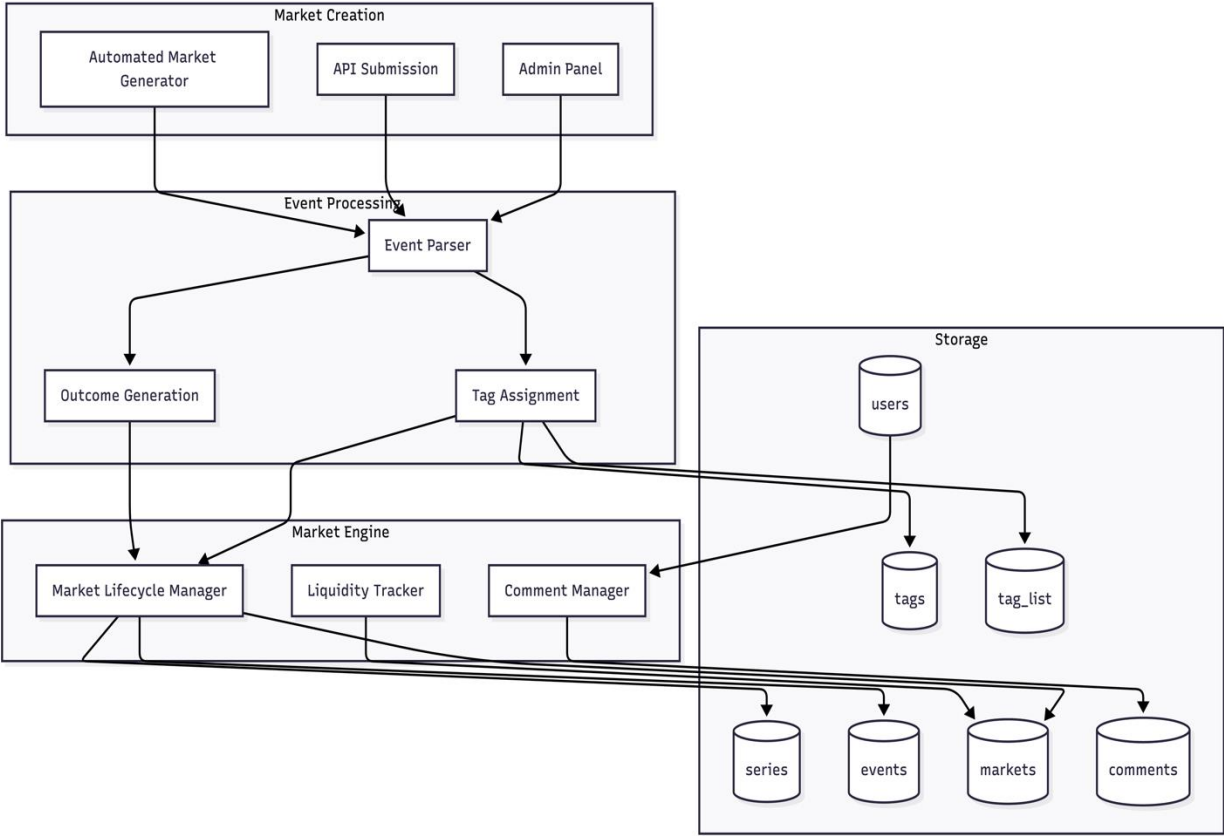


Architecture/pipeline

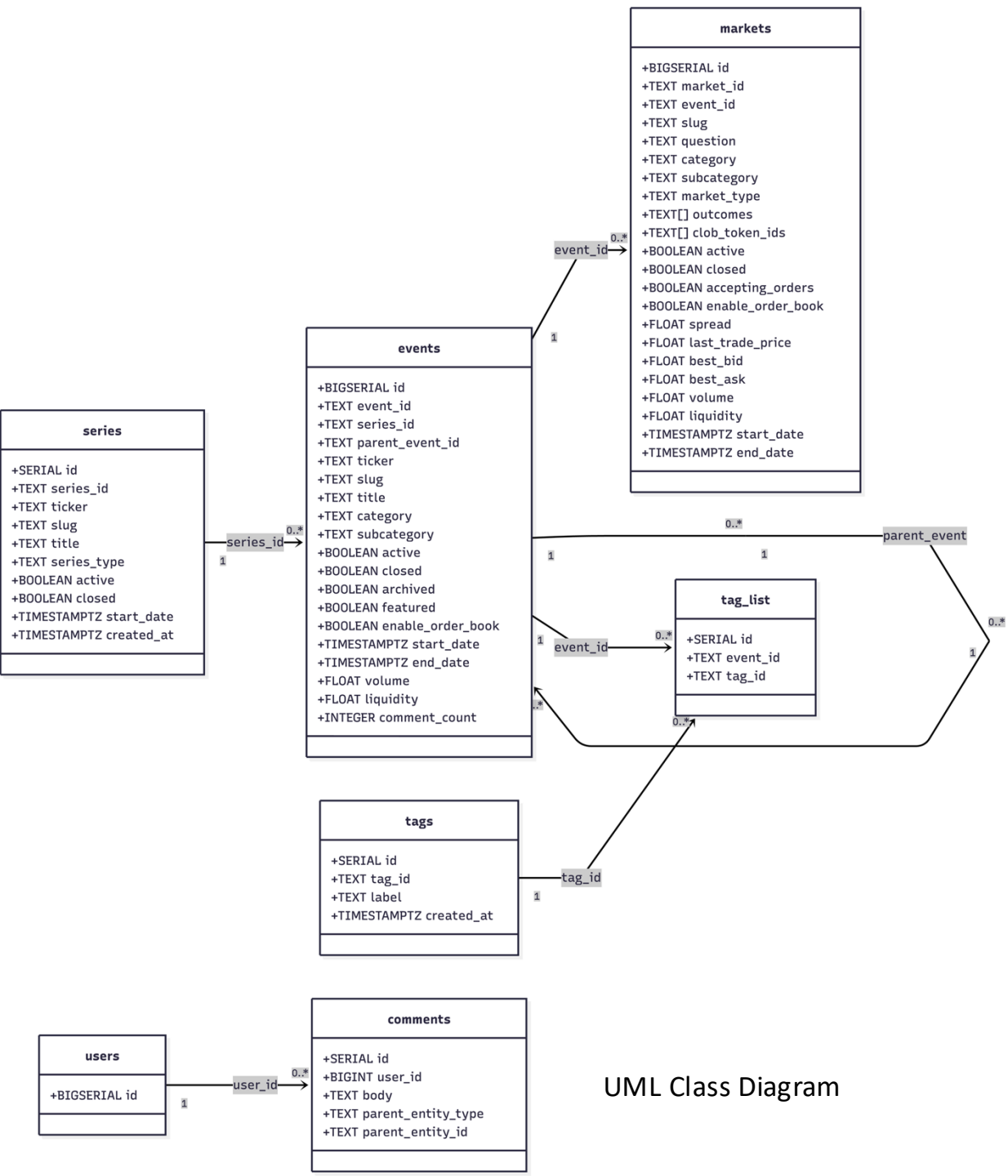


UML Class Diagram

UML Class Diagram - Market



Architecture/pipeline



UML Class Diagram

Class design

Trading/Order Book System

- Core class: token, order_book_snapshot, order_book_update, trade, user
- Relations:
 - Token -> order_book_snapshot
 - Token -> order_book_update
 - Token -> trade
 - User -> trade

Price Oracle System

- Core class: price, binance_price, chainlink_price
- Relations: price --> binance_price / chainlink_price

Market Structure System

- Core class: series, events, markets, tags, tag_list, comments, users
- Relations:
 - series -> events -> markets
 - events -> events (parent_event)
 - events -> tag_list -> tags
 - users -> comments

Usage

Store the order book state and trade executions for each token.

Provides external reference prices from different oracle sources.

Defines the hierarchical structure of prediction markets and their metadata.

Thank you!